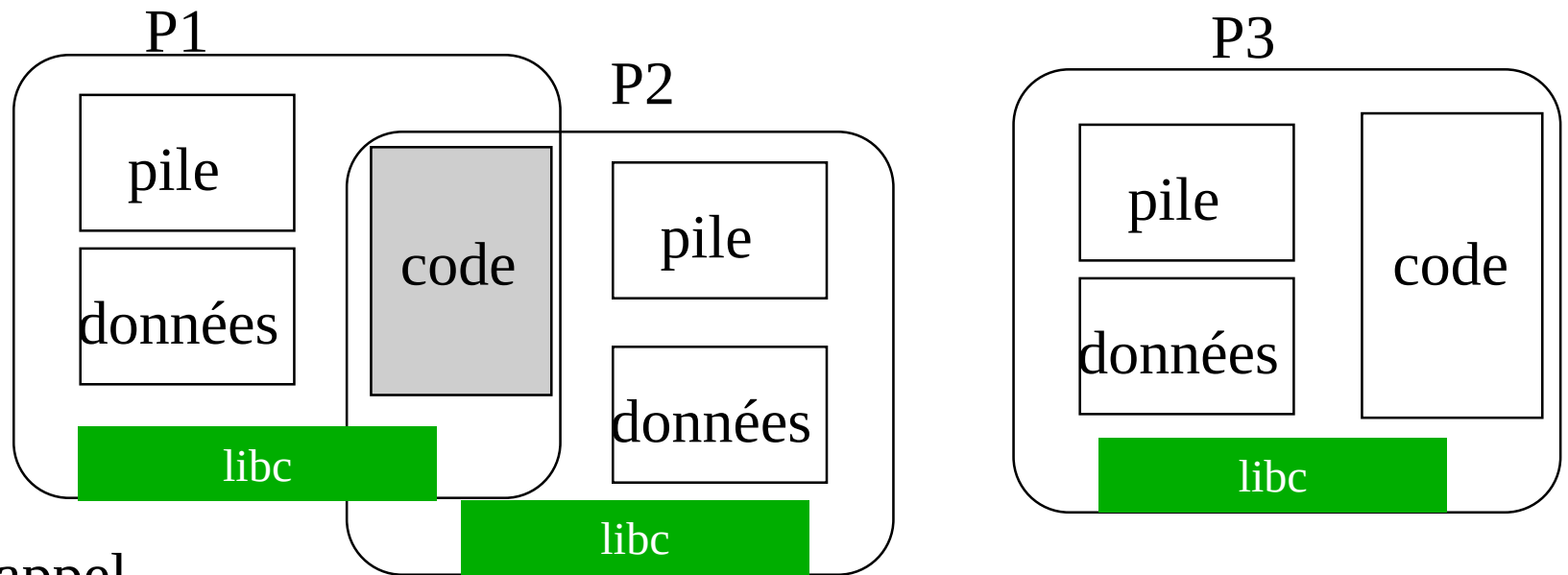


Gestion de processus

1. Architecture - Mode d'exécution- Etats
2. Interruption / Synchronisation
3. Signaux
4. Ordonnancement
5. Appels système : Création/termination
6. Processus légers – threads
7. Linux
8. Windows

Architecture

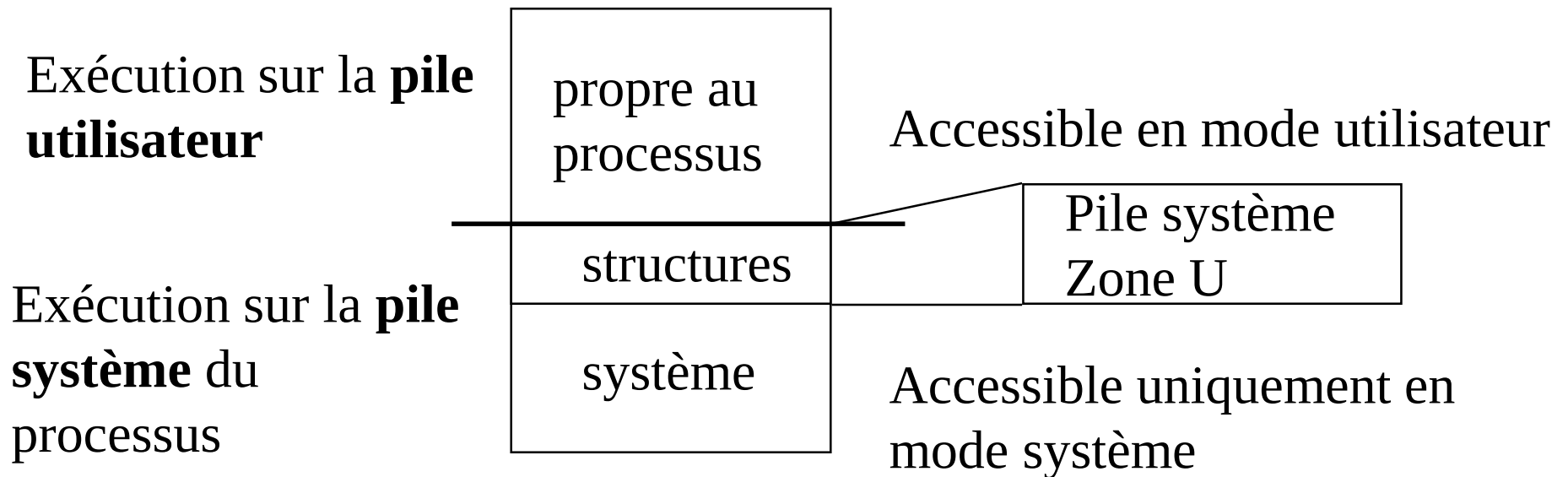


Interface appel
système

Noyau

Mode d'exécution

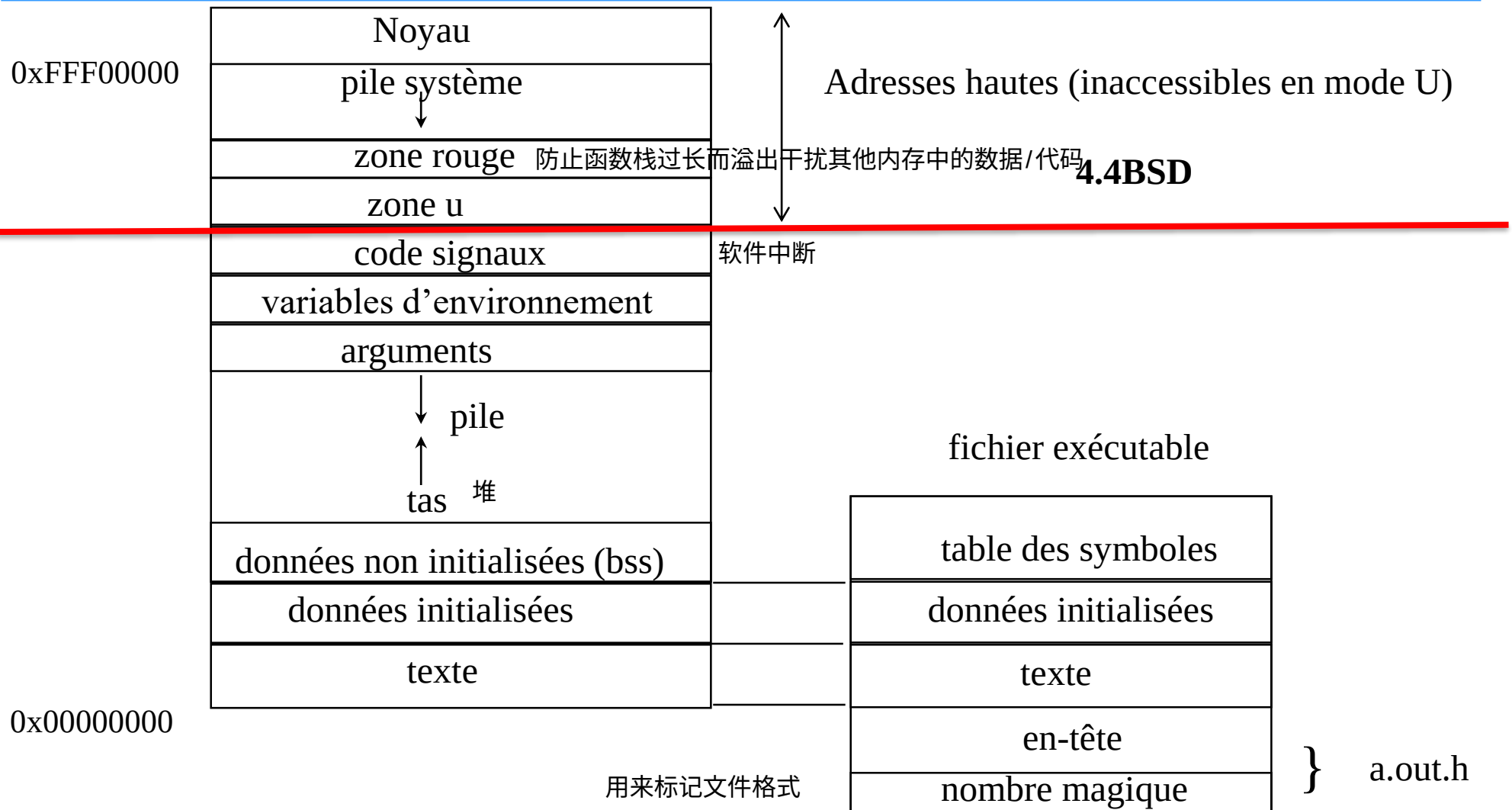
- Deux modes : utilisateurs / systèmes
 - OS/2 3 niveaux, Multics 7
- => 2 zones de mémoire virtuelle :
 - le noyau fait partie de l'espace virtuel du processus courant !



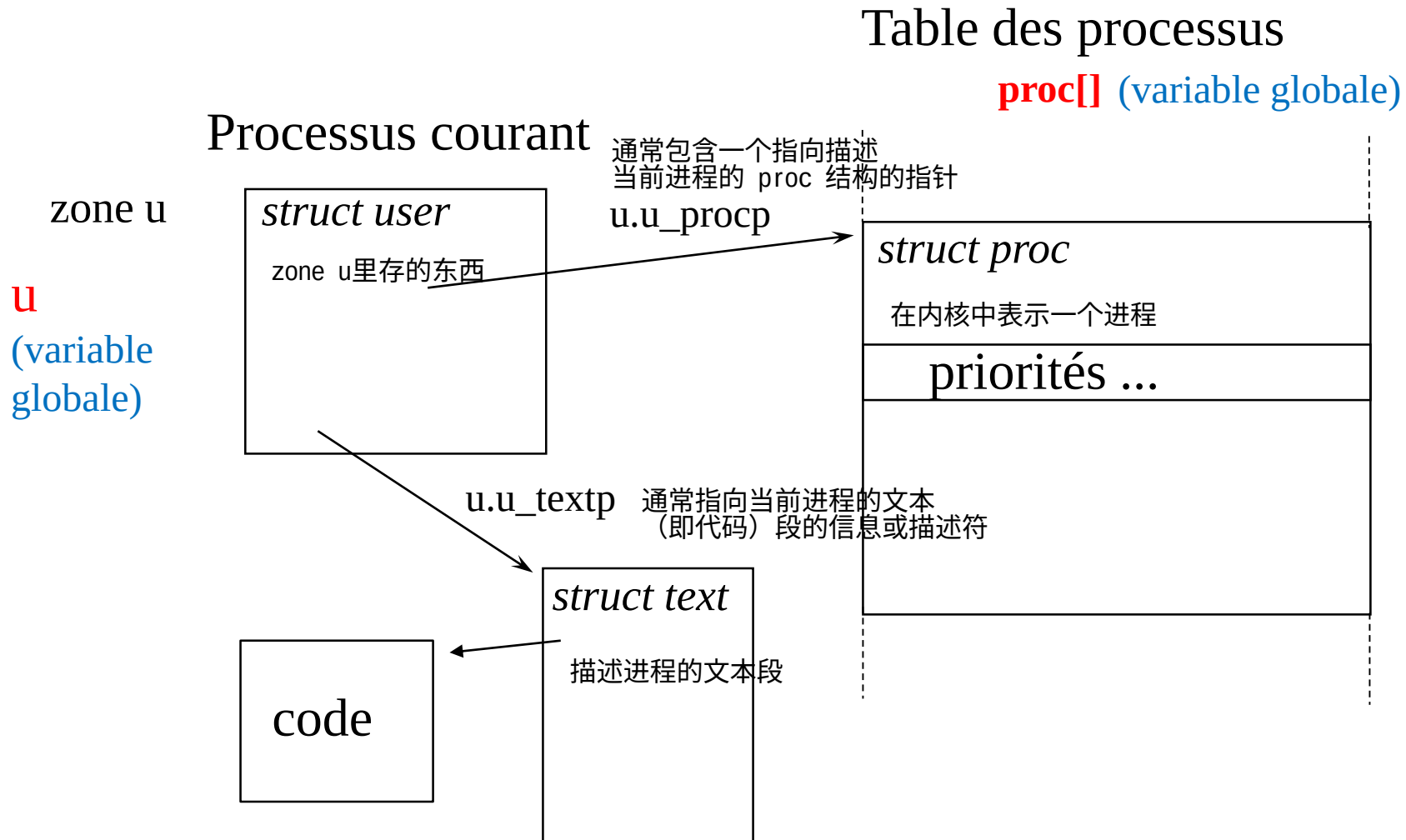
Structure interne

- Contexte :
 - Espace utilisateur (données, pile)
 - Information de contrôle (zone u, struct proc)
 - Variables d'environnement
- Contexte matériel :
 - Compteur ordinal (PC)
 - Pointeur de pile
 - Mot d'état (Process Status Word) : état du système, mode d'exécution, niveau de priorité d'interruption
 - Registre de gestion mémoire
 - registres FPU (Floating point unit)
- Commutation => sauvegarde du contexte mat. dans zone u (pcb : process control bloc)

Processus en mémoire et sur disque



Les structures en mémoire



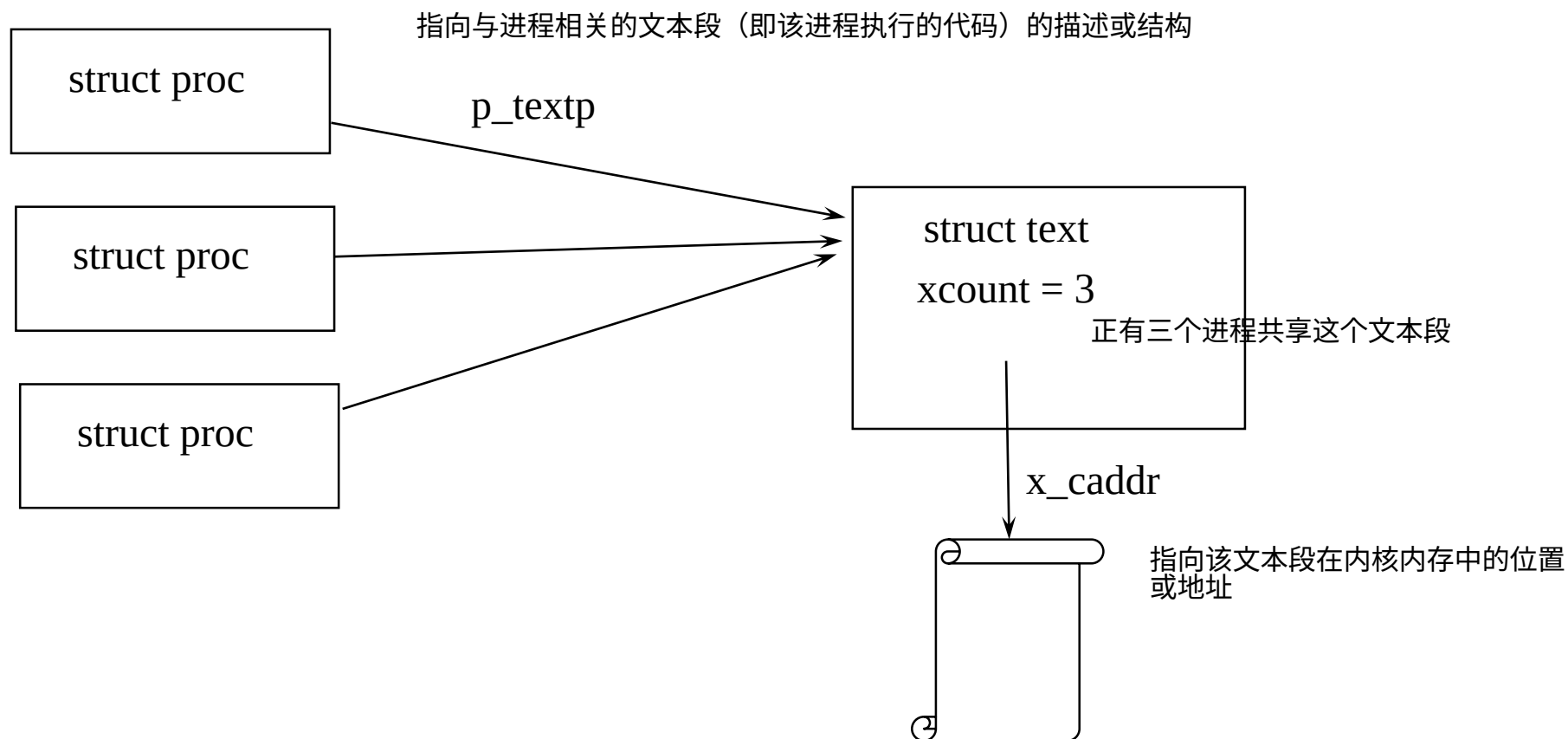
Structure user

- struct user (user.h)
- Fait partie de l'espace du processus => swappable
 - pcb
 - pointeur vers struct proc
 - uid et gid effectif et réel
 - arguments, valeurs de retour , erreurs de l'appel système courant
 - information sur les signaux
 - entête du programme
 - table des fichiers ouverts
 - pointeurs vers vnodes du répertoire courant, terminal
 - statistiques d'utilisation CPU, quotas, limites
 - [pile système]

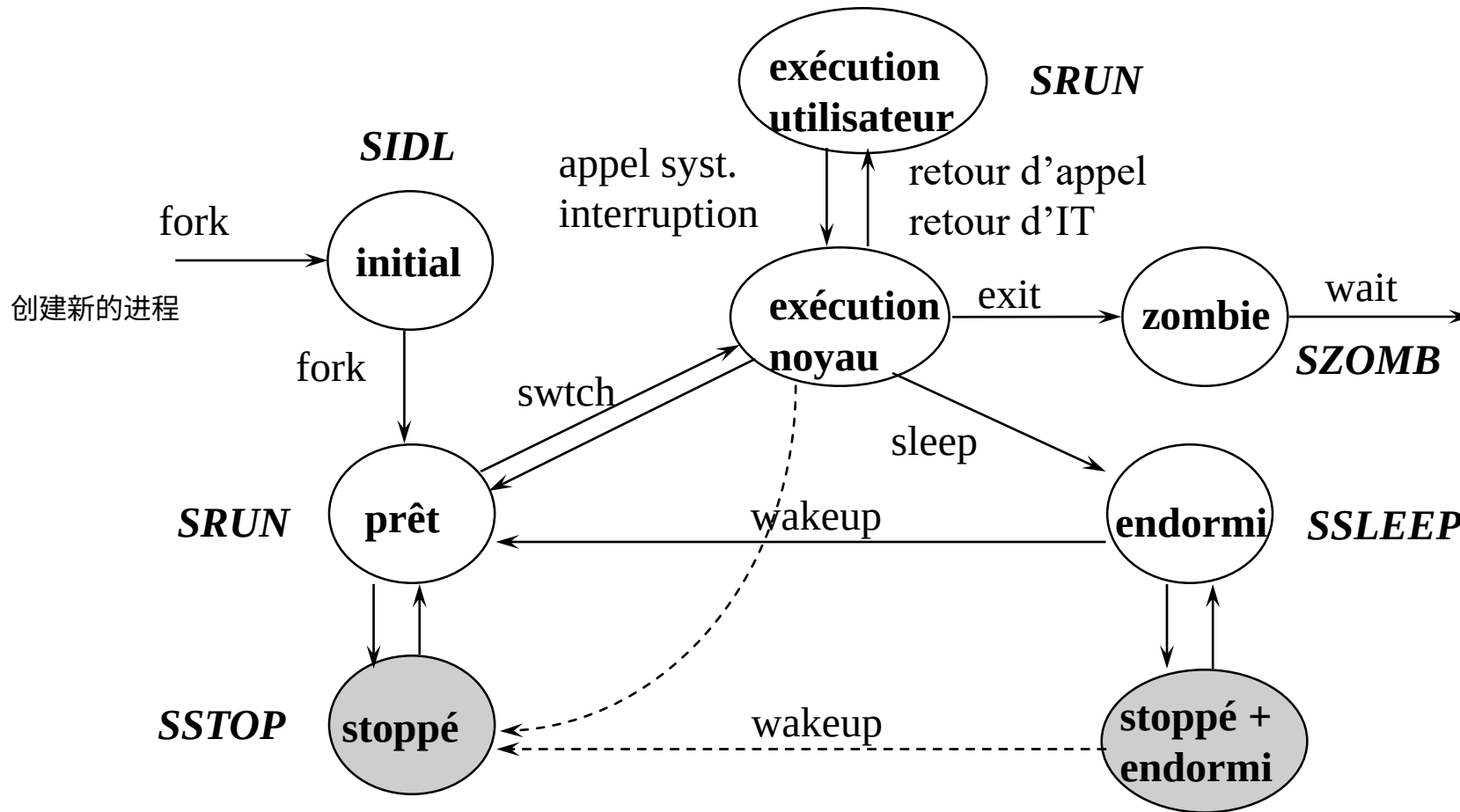
Structure résidente

- struct proc (proc.h)
 - pid, gid
 - pointeur zone U
 - état du processus
 - pointeurs vers liste de processus prêts, bloqués ...
 - événement bloquant
 - priorité + information d'ordonnancement
 - masque des signaux
 - information mémoire
 - pointeurs vers listes des processus actifs, libres, zombies

Partage de code



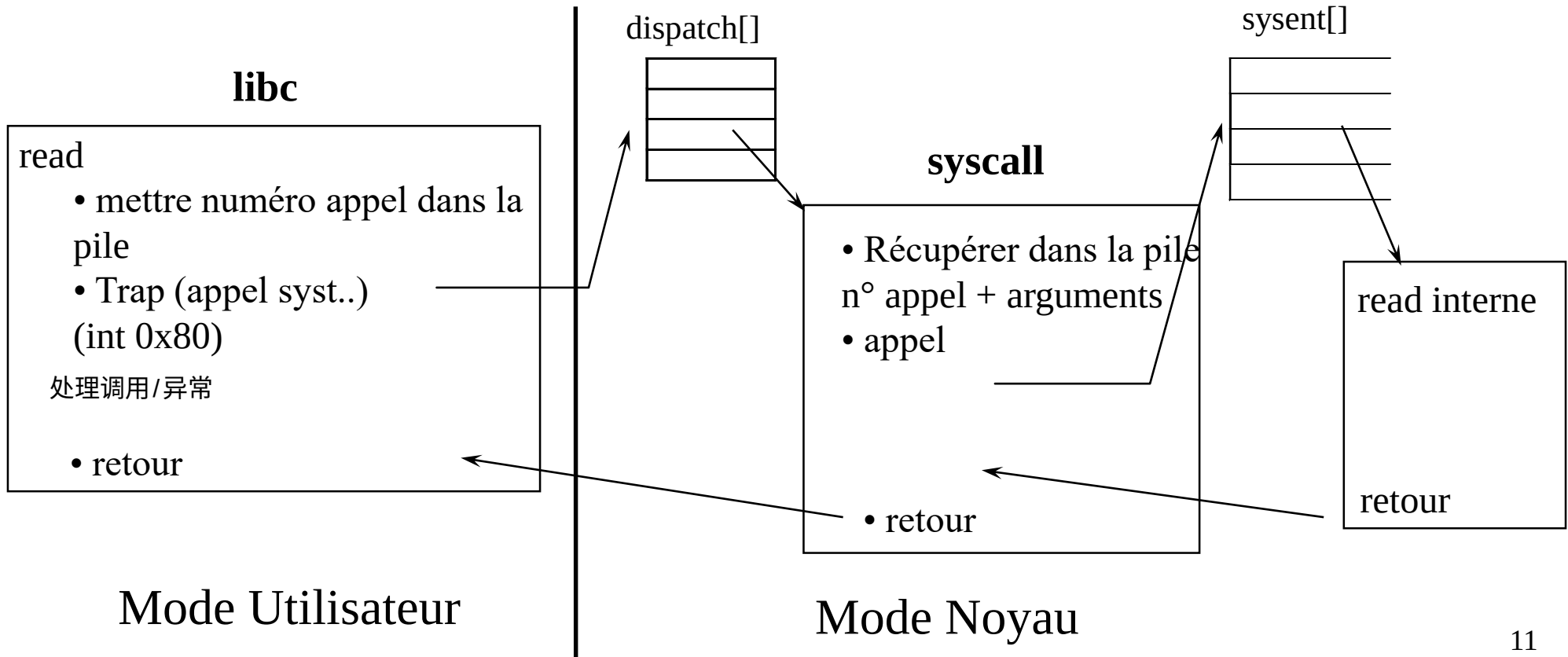
Etat d'un processus



BSD

Interface des appels systèmes

- Appels système encapsulés par des fonctions de librairie
- Chaque appel est identifié par un numéro



Algorithme de syscall

- Trouver les paramètres dans la pile du processus
- Copier les paramètres dans zone U (champs u_arg)
- Sauvegarder le contexte en cas de retour prématuré (interruption par des signaux)
- Exécuter l'appel
- Si erreur : positionner le bit report du mot d'état
mettre le numéro d'erreur dans un registre
- Au retour de l'appel tester le bit report

syscall – systèmes récents

- Utilisation interruption couteux (indirection).
- Depuis 1997 (Intel), 1998 (AMD) : *Fast system calls*
 - sysenter/sysexit (Intel32)
 - syscall/sysret (AMD, Intel64)
- Intel :
 - 1 registre **MSR** stocke l'adresse de la fonction syscall
 - sysenter => exécution en mode privilégié de la fonction désignée par MSR

Exemple appels sur intel

```
.data
.set .L_STDOUT,    1                intel32
.set .L_SYSCALL_EXIT, 1
.set .L_SYSCALL_WRITE, 4
.L_message:
.ascii "Hello, world!\n"
.set .L_message_len, . - .L_message

.text
.global _start
_start:

    # write(STDOUT, message, message_len)
    mov $.L_SYSCALL_WRITE, %eax
    mov $.L_STDOUT,    %ebx
    mov $.L_message,    %ecx
    mov $.L_message_len, %edx
    int $0x80

    # exit(0)
    mov $.L_SYSCALL_EXIT, %eax
    mov $0,            %ebx
    int $0x80
```

```
.data
.set .L_STDOUT,    1                intel64
.set .L_SYSCALL_EXIT, 60
.set .L_SYSCALL_WRITE, 1
.L_message:
.ascii "Hello, world!\n"
.set .L_message_len, . - .L_message

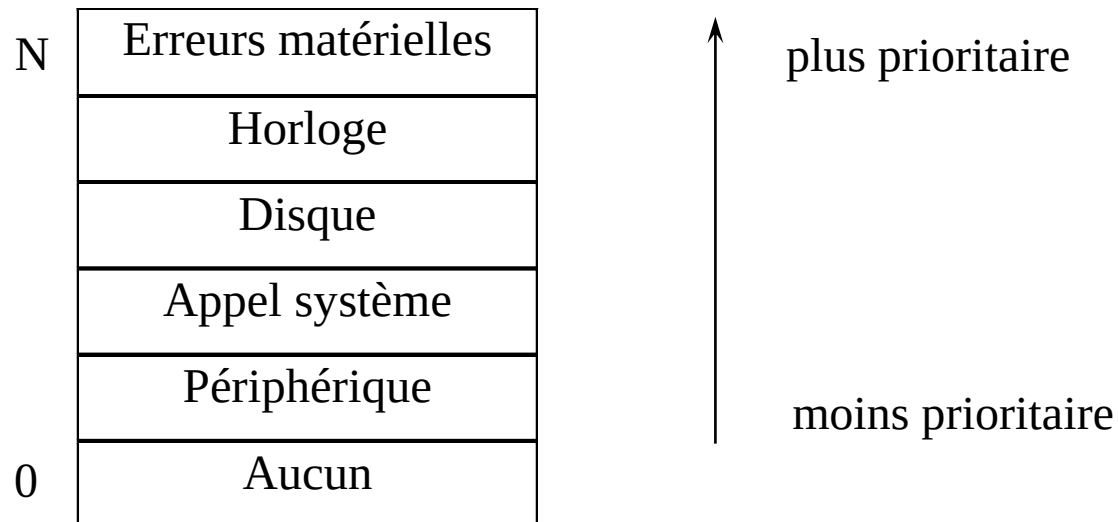
.text
.global _start
_start:

    # write(STDOUT, message, message_len)
    mov $.L_SYSCALL_WRITE, %rax
    mov $.L_STDOUT,    %rdi
    mov $.L_message,    %rsi
    mov $.L_message_len, %rdx
    syscall

    # exit(0)
    mov $.L_SYSCALL_EXIT, %rax
    mov $0,            %rdi
    syscall
```

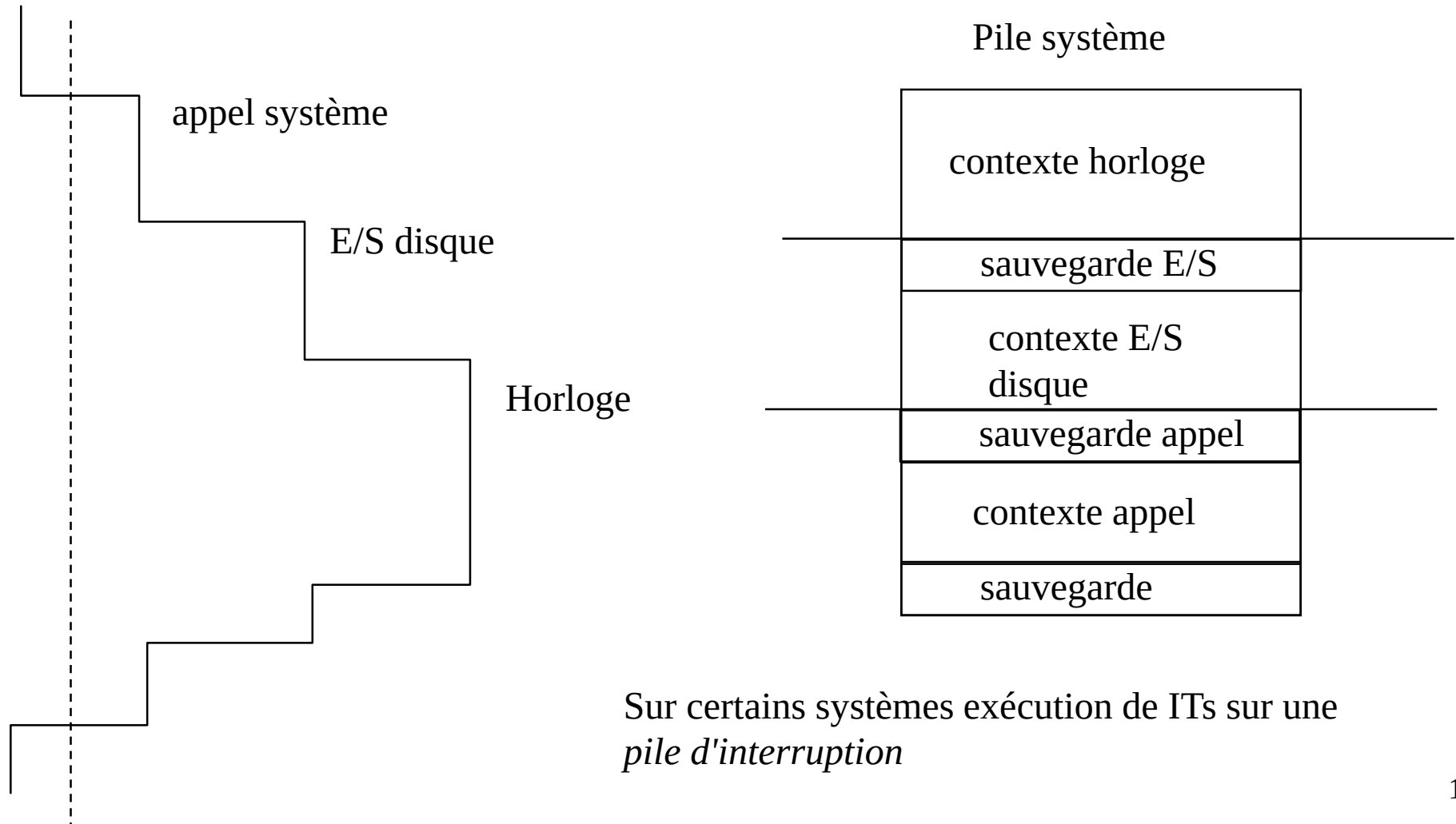
Les interruptions

- Pour chaque interruption, un niveau de priorité (ipl: interrupt priority level)
- 7 niveaux Unix de base, 32 niveaux Unix BSD

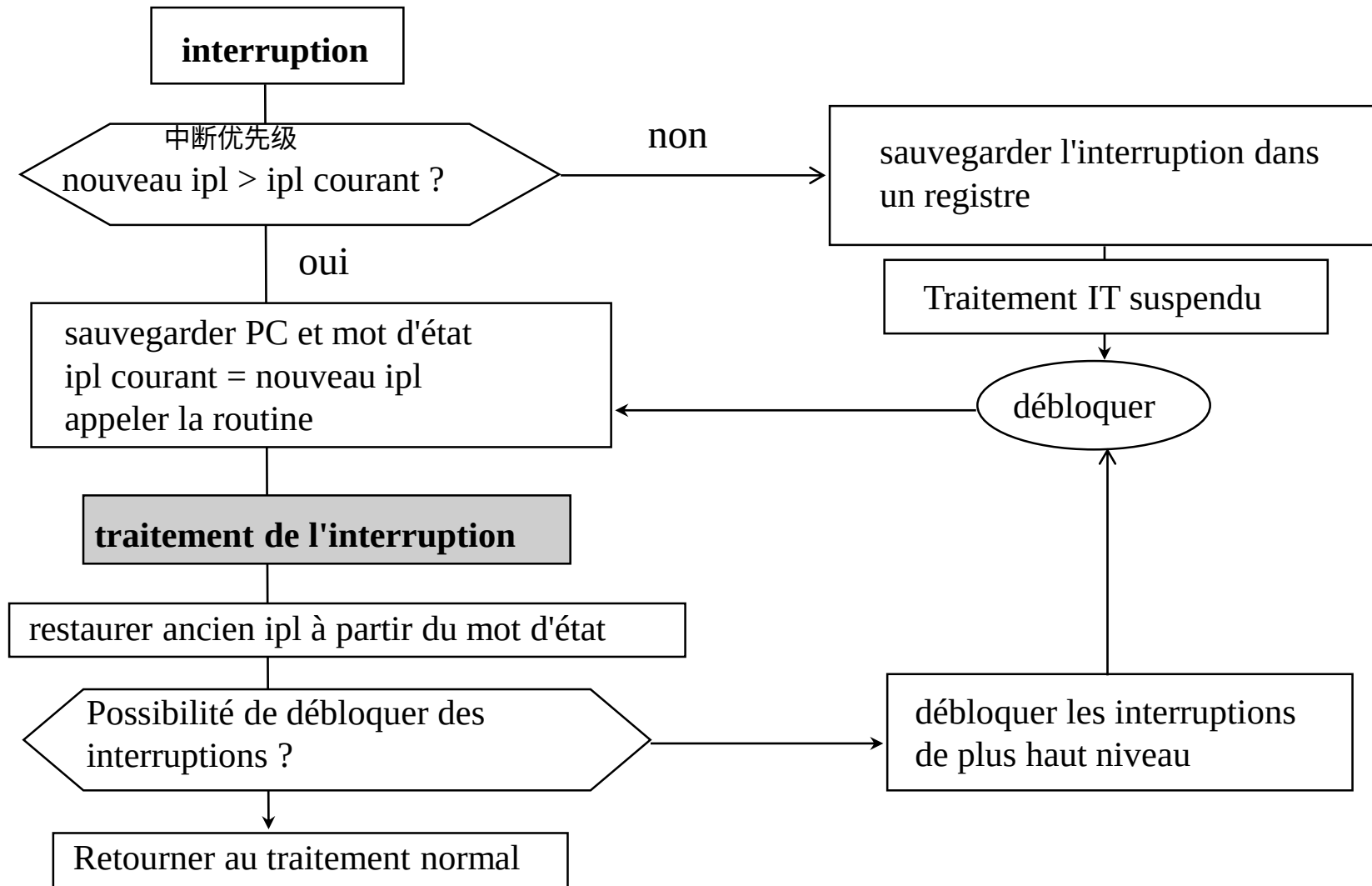


- ipl stocké dans le mot d'état

Les niveaux d'exécution



Traitement des interruptions



Synchronisation

- Unix est **ré-entrant** => A un instant donné, plusieurs processus dans le noyau :
 - un seul est cours d'exécution
 - plusieurs bloqués
- Problème si manipulation des mêmes données
 - nécessité de protéger l'accès aux ressources
 - => noyaux **non préemptifs** : Un processus s'exécutant en mode noyau ne peut être interrompu par un autre processus *sauf blocage explicite*
 - => 1) synchronisation uniquement pour les opérations bloquantes
 - ex: lecture d'un tampon => verrouillage du tampon pendant le transfert
 - 2) possibilité d'interruption par les périphériques => définition de section critique

Section critique

- Appel à set-priority-level pour bloquer les interruptions (bsd)

Primitive (bsd)	Activité bloquée	Unix TD
slp0()	aucune	spl(NORMAL)
splsoftclock()	horloge faible priorité	
splnet()	protocoles réseaux	
spltty()	terminaux	
splbio()	disques	spl(BDINHB)
splimp()	périphériques réseaux	
splclock()	horloge	spl(CLINHB)
splhigh()	toutes les interruptions	

- Exemple :

```
s=splbio(); /*augmenter la priorité pour bloquer les IT disques */
```

```
...
```

```
splx(s); /* restaurer l'ancienne priorité */
```

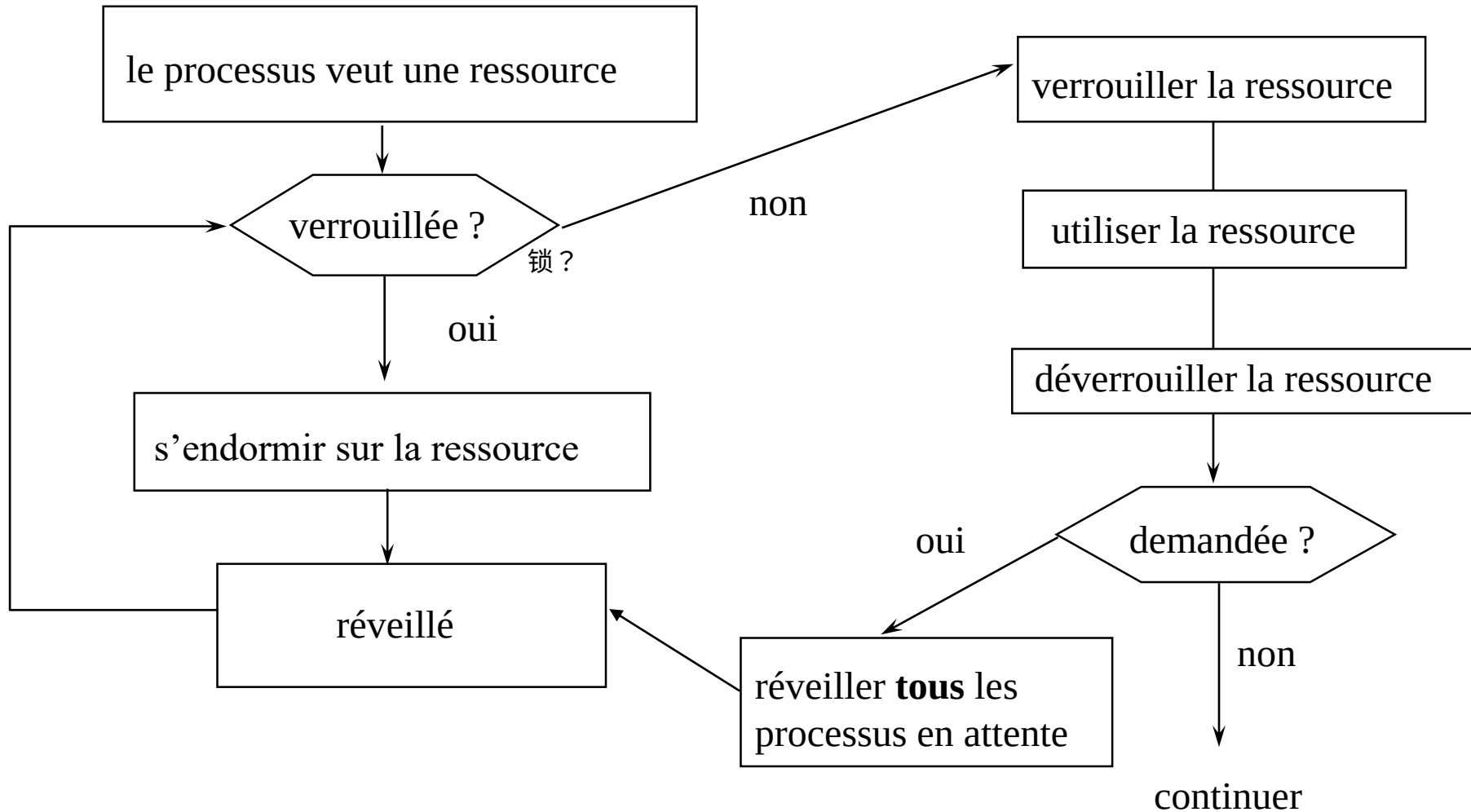
Unix TD :

```
s=gpl(); spl(BDINH);
```

```
...
```

```
spl(s);
```

Verrouillage de ressources



Exemple de code

- Verrouillage

/* Attente d'une E/S sur bp */

```
iowait(bp) {  
    slp(BDINHB);  
    while ( !bp->b_flags & B_DONE )  
        sleep(bp, PRIBIO);  
    slp(NORMAL);  
    ...  
}
```

/* Fin d'E/S */

```
iodone(bp) {  
    ...  
    bp->b_flags |= B_DONE ;  
    if (bp->b_flags & B_ASYNC)  
        brelse(bp) ;  
    else wakeup(bp) ;  
}
```

iodone exécuter ici => sleep inutile !

Primitive sleep

- 2 paramètres :
 - adresse de l'obstacle
 - Priorité en mode noyau (priorité du processus endormi)

- Priorité (4.3BSD):

- | | |
|----------|----------------------------------|
| – PSWP | Swapper |
| – PMEM | Démon de pagination |
| – PINOD | Attente d'une inode |
| – PRIBIO | Attente E/S disque |
| – PZERO | Seuil |
| – PPIPE | Attente sur tube (plein ou vide) |
| – TTIPRI | Attente entrée sur un terminal |
| – TTOPRI | Attente écriture sur un terminal |
| – PWAIT | Attente d'un fils |
| – PSLEP | Attente d'un signal |

Non interruptibles par des signaux

Interruptibles par des signaux

Algorithme de sleep

- **Masquer** les interruptions
- Mettre le processus à l'état SSLEEP
- Mise à jour du champs p_wchan (obstacle)
- Changer le niveau de priorité du processus
- Si (priorité non interruptible) {
 - commutation (swtch) /* le processus dort */
 - /* réveil */
 - démasquer les interruptions
 - retourner 0}
- /* priorité interruptible */
- Si (pas de signaux en suspens) {
 - commutation (swtch) /* le processus dort */
 - Si (pas de signaux en suspens) {
 - démasquer les interruptions /* Pas réveillé par un signal */
 - retourner 0;}}
- /* Signal reçu ! */
- **Démasquer** interruptions
- restaurer le contexte sauvegardé dans appel système
- saut (longjmp)

Algorithme de wakeup

- Réveiller tous les processus en attente sur l'obstacle
 - Masquer interruption
 - pour (tous les processus endormis sur l'obstacle) {
 - mettre à l'état prêt
 - si (le processus n'est pas en mémoire)
 - réveiller le swapper
 - sinon si(processus plus prioritaire que processus courant)
 - marquer un flag (runrun, Unix TD)
 - }
 - Démasquer interruptions
- Retour en mode utilisateur => test du flag :
 - Si (flag positionné) réordonner

Initialisation du système

- Initialisation des structures :

空闲inode列表 — liste des inodes libres, table des pages 虚拟地址到物理地址的转换

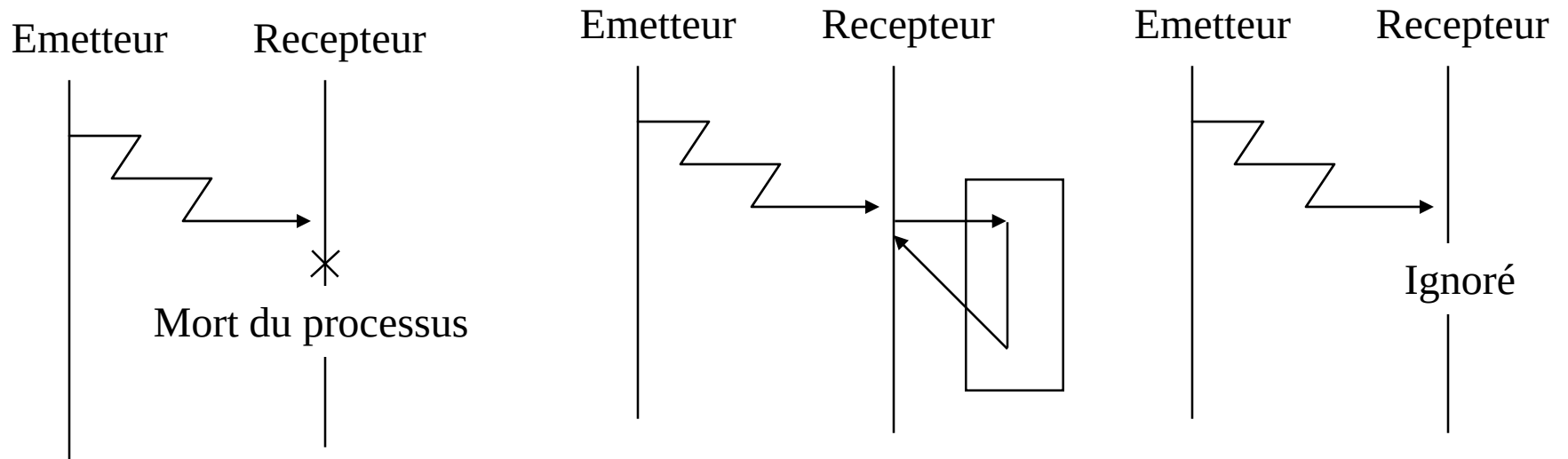
- montage de la racine
- construire le contexte du processus 0
(struct U, initialisation de proc[0])
- Fork pour créer le processus 1 (init)
- Exécuter le code du swapper (fonction sched)

Processus du système

- Processus 0 : swapper
gère le chargement/déchargement des processus sur le swap
- Processus 1 : init lance les démons d'accueil (gettyd)
- Processus 2 : paginateur (pagedaemon) - gère le remplacement de pages
- Autres "démons" :
inetd, nfsd, nfsiod, portmapper,....

Les signaux

- Mécanisme de communication de base
- Principe :
 - 1 signal Unix = 1 entier



Les principaux signaux

– SIGHUP	1	Deconnexion d'un terminal
– SIGINT	2	Ctrl-C
– SIGQUIT	3 *	Caractère <QUIT> frappé au clavier
– SIGILL	4 *	instruction illégale
– SIGTRAP	5 *	Tracer le processus
– SIGIOT	6 *	Problème du matériel
– SIGKILL	9	Tuer un processus
– SIGBUS	10 *	Erreur sur le bus
– SIGSEGV	11 *	Violation de segment mémoire
– SIGSYS	12 *	Erreur paramètres dans appel système
– SIGPIPE	13	Erreur ecriture sur un tube ou socket sans lecteur
– SIALRM	14	Horloge
– SIGSTOP	17	Stoppe l'exécution
– SIGCONT	19	Continuer l'exécution
– SIGCHLD	20	Envoyer au père en fin d'exécution
– SIGUSR1	30	Réservé à l'utilisateur
– SIGUSR2	31	" "

* :génère un core

Primitives de base

- Envoi d'un signal:
 - `int kill (int pid, int signal)`
Par défaut la réception d'un signal provoque la terminaison
- Définition de la routine de traitement
 - `signal(int signal, void (*fonc)(int signal));`
fonc : fonction appelée lors de la réception du signal
- 2 fonctions prédéfinies :
 - `SIG_IGN` (Ignore le signal)
 - `SIG_DFL` (Restituer le comportement par défaut)
- Applicable à tous les signaux sauf `SIG_KILL`
- Attendre un signal quelconque:
 - `int pause(void)`

Exemple

```
#include <signal.h>
```

```
void tsignal(int sig) {  
    printf("signal numéro %d reçu \n", sig);  
    signal(sig, tsignal);    // Inutile sous certains systèmes  
}
```

```
main() {  
    signal(SIGINT, tsignal);  
    signal(SIGUSR1, tsignal);  
    sleep(100);  
}
```

Attention : ne pas utiliser de fonctions non réentrantes (ex. malloc, free ...)

Utilisation des temporisateur

- `alarm(int sec);`
 - Génère SIGALRM au bout de "sec" secondes

```
#include <signal.h>
int nb_sec = 0;
void seconde(int n) {
    alarm(1);
    signal (SIGALRM, seconde);
    nb_sec++;
}
void tsigurs1(int s) {
    printf("%d secondes écoulée \n", nb_sec);
}
main() {
    alarm(1);
    signal(SIGALRM, seconde); signal(SIGUSR1, tsigusr1);
    while (1)
        pause();
}
```

Gestion des signaux

- Structures

- Dans la zone U :

- `u_signal[]` routines de traitements

SVR4

- `u_sigmask[]` masque associé à chaque routine

- Dans struct proc :

- `p_sig` signaux en cours

SVR4

- `p_cursig` masque des signaux “pendants”
 - `p_sig` signal en cours de traitement
 - `p_hold` masque des signaux bloqués
 - `p_ignore` masque des signaux ignorés

Signaux : Génération

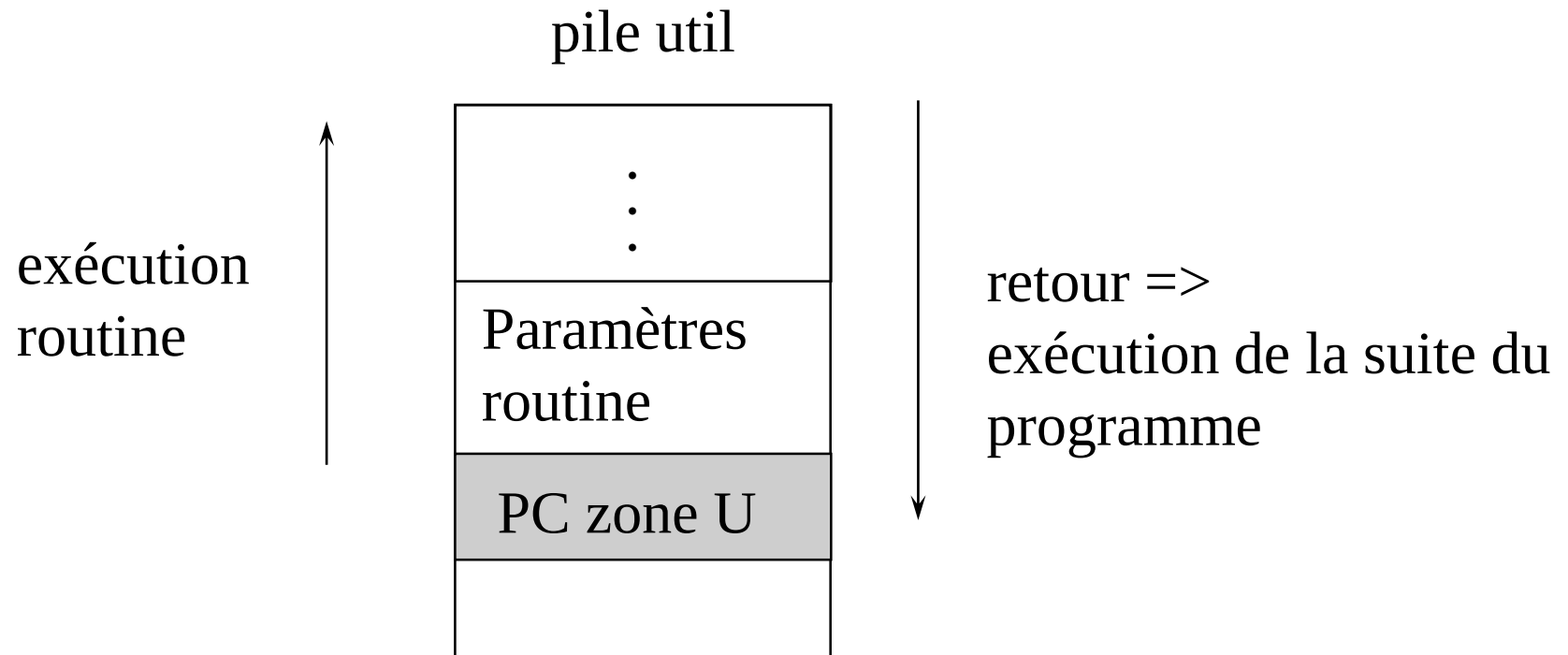
- Lors d'un "kill"
 - Chercher la structure proc du processus cible
 - Ajouter le signal dans p_sig
 - Si le processus est bloqué dans le noyau, le réveiller (rendre prêt)
- => 1 seul traitement pour plusieurs instances du même signal
- Le signal ne sera traité que lorsque le processus cible passera sur le processeur

Signaux : traitement (1)

- Vérifier la présence de signaux : appel à issig
 - issig est appelée lors : retour au mode utilisateur (après appel système ou interruption)
 - issig :
 - Trouver un signal positionné dans p_sig
 - Retirer les signaux ignorés
 - Retourner numéro de signal, 0 si aucun signal
- Si issig retourne true (!=0) traiter le signal : appel de psig
 - psig:
 - Trouver la routine de traitement dans u_signal du processus courant
 - Si aucune routine exécuter le traitement par défaut
 - (...p_hold |= ...u_sigmask)
 - Appel de sendsig qui exécute la routine lors du retour en mode util.

Signaux : traitement (2)

- sendsig : appel dépendant de la machine



Ordonnancement

1. Interruption horloge
2. Les structures
3. Ordonnanceurs classiques (BSD, SVR3)
4. Classes d'ordonnancement (SVR4)
5. Ordonnancement temps réel (SVR4, Solarix 2.x)

Les horloges matérielles

- RTC : Real-Time Clock
 - Horloge temps-réel
 - Maintenue par batterie lorsque l'ordinateur est éteint
 - Précision limitée, accès lent
 - Utilisée au démarrage pour mettre à jour l'horloge système
- TSC : Time Stamp Counter
 - Compteur 64 bits (Intel)
 - Incrementé à chaque cycle horloge
 - ex: 1G HZ => incrémentation toutes les ns ($1/1E9$) => sur 64 bits débordement au bout de 584 ans !
 - Mesure précise du temps
 - Mesure directement dépendante de la fréquence du processeurs => pb avec portable
- PIT : Programmable Interval Timer
 - Registre horloge => agit comme un minuteur
 - Décrémentation régulière, Passage à 0 => interruption horloge ITH (IRQ0)
 - Outils de base de l'ordonnanceur
 - Précision de 100 HZ (10 ms) sur la plupart des UNIX – 1000 HZ (1 ms) dans certain linux 2.6

Interruption horloge : `hardclock()`

- Horloge matérielle interrompt le processus à des intervalles de temps fixes = tics horloge
- tic 10 ms
 - HZ dans `param.h` indique le nombre de tics par seconde (par ex. 100 Linux, 60 Unix V6)
- Routine de traitement dépendant du matériel
- Doit être courte !
- Très prioritaire
- 1 quantum = 6 ou 10 tics

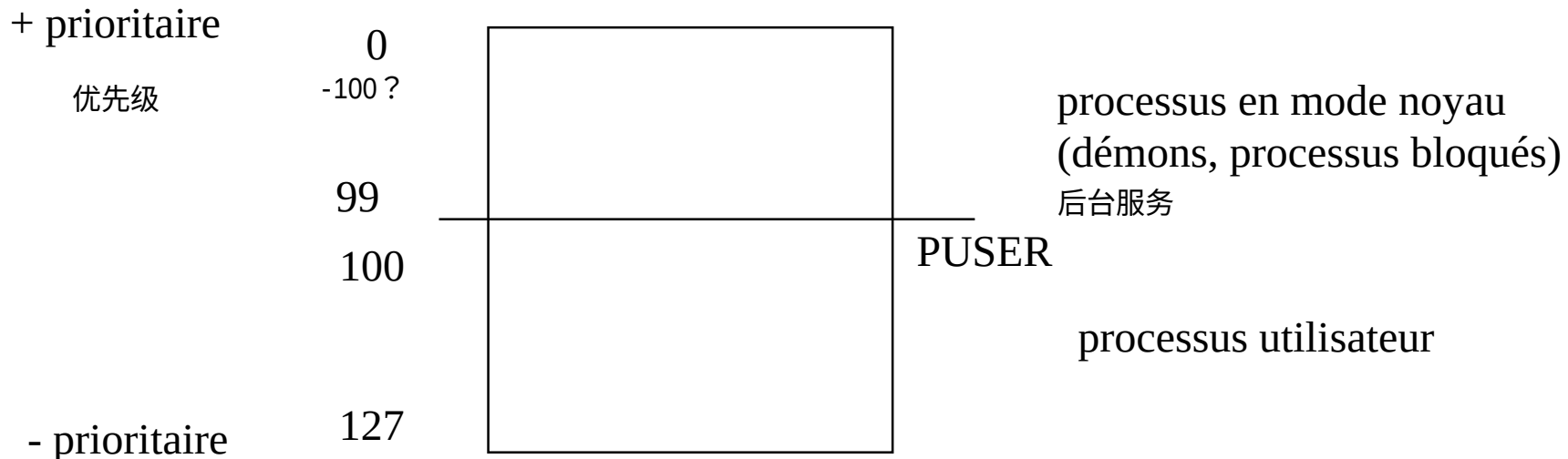
Routine de traitement de IT horloge

1. Réarmer l'interruption horloge
2. Mise à jour des statistiques d'utilisation CPU du processus courant (p_cpu)
3. Recalculer la priorité des processus
4. Traiter fin de quantum
5. [Envoyer SIGXCPU au processus courant si quota CPU dépassé]
6. Mise à jour de l'horloge
7. Réveiller processus système si nécessaire
8. Traiter les alarmes

Structures

- Ordonnancement basé sur les priorités
- Les informations sont stockées dans struct proc (résident)

- p_pri Priorité courante 当前优先级
- p_cpu mesure de l'activité CPU récente CPU使用情况或活跃度
- p_nice incrément de priorité contrôlable par l'utilisateur 优先级增量值
p_nice越高, 优先级越低



Ordonnancement classique

- Répartir équitablement le processeur =>
 - baisser la priorité des processus lors de l'accès au processeur
- 每个时钟中断/滴答 A chaque tic p_cpu++ pour le processus courant
- Régulièrement appel de `schedcpu()` (1 fois par seconde)
 - Pour tous les processus prêts : 更新进程cpu使用信息

$$p_pri = PUSER + p_cpu/4 + 2*p_nice$$

p_cpu逐渐降低 $p_cpu = p_cpu * decay$

$$decay = 1/2$$

System V Release 3

p_cpu随负载增大而减小 $decay = (2 * load) / (2*load + 1)$ BSD

- Un processus qui a eu un accès récent => p_cpu élevé => p_pri élevé.
- 调整p_nice **setpriority()** ou **setpri()** pour mettre à jour la priorité du processus courant.

Exemple - System V Release 3

- Quantum = 60 tics, PUSER = 60
- Fin Quantum : $p_cpu = p_cpu / 2$; $p_pri = PUSER + p_cpu/2$

A		B		C	
p_pri	p_cpu	p_pri	p_cpu	p_pri	p_cpu
60	0 60	60	0	60	0
75	30 30	60	0 60	60	0 0
67	15 15	75	30 30	60	0 60
63	7	67	15	75	30

Fin Q

42

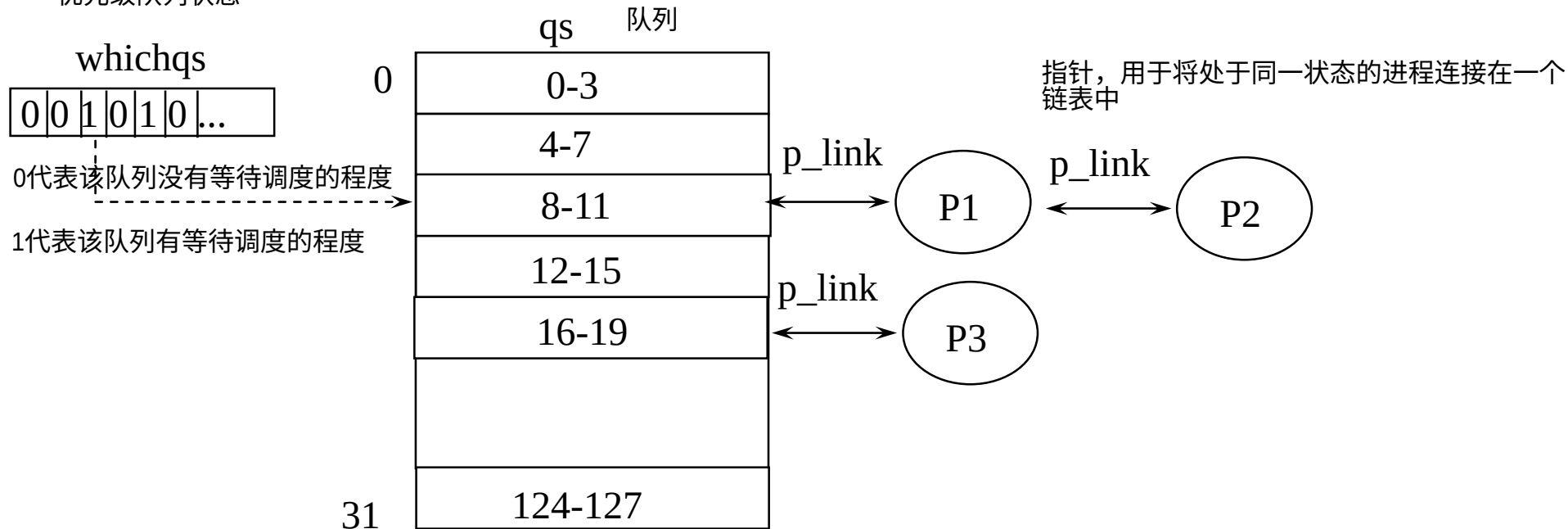
Priorité des processus bloqués

- Les processus sont bloqués avec une haute priorité $\neq$ priorité utilisateur
=> Au réveil le processus a une plus grande probabilité d'être élu
- Au passage au mode U la priorité de niveau utilisation est recalculée

Implémentation

- Problème : trouver **rapidement** le processus le plus prioritaire
- BSD : 32 files de processus prêts

优先级队列状态

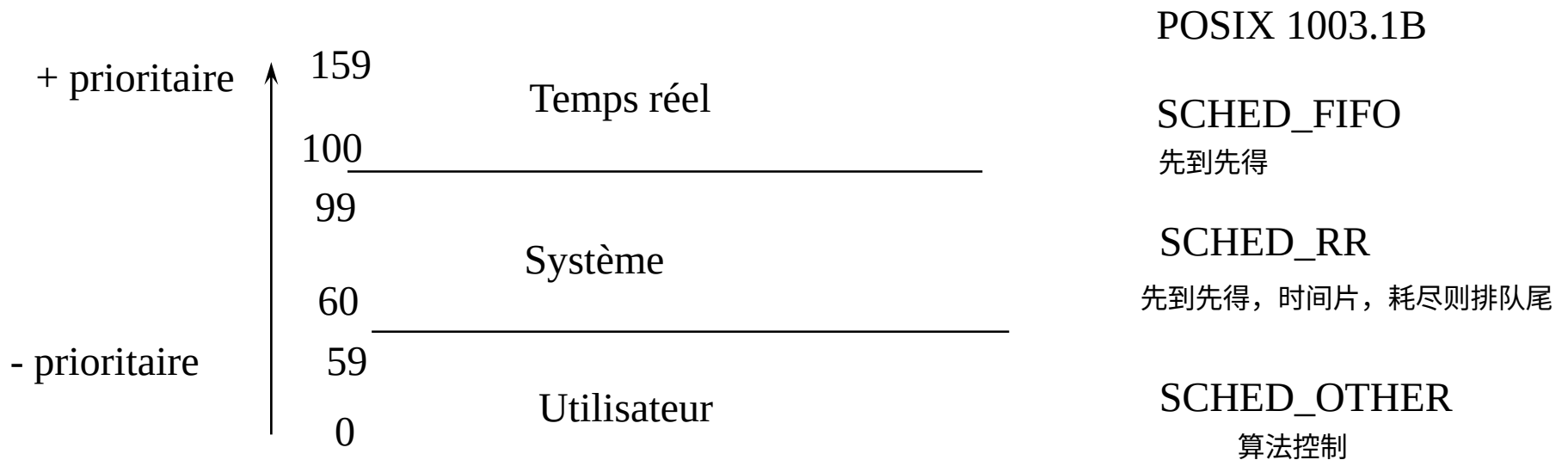


Algorithme de swtch

- Trouver le premier bit positionné dans whichqs whichqs中找1
 - Retirer le processus le plus prioritaire de la tête 从1的队列头部找出最高优先级
 - Effectuer la commutation :
 - Sauvegarder les registres (le PCB = Process Control Bloc) du processus courant dans sa structure user.
- [*Unix Mémoire virtuelle* :
Changer la valeur du registre de la table des page]
- Copier la "struct user" du nouveau processus dans **u**
 - Charger les registres du processus élu à partir de la zone u

SVR4 : Classes d'ordonnancement

- 3 classes de priorités



- Les processus temps réel prêt s'exécutent tant qu'ils restent prêt
- Définition des processus temps réel réservée au superviseur (appel système priocntl SVR5 - sched_setparam POSIX)

SVR4 : Structures

- Ajout dans struct proc :
 - p_cid : identificateur de la classe ...
- Une liste des processus temps réel (rt_plist)
- Une liste de processus temps partagé (ts_plist) ...

Classe "temps partagé"

- Quantum variable d'un processus à l'autre 时间片可变
- inversement proportionnel à la priorité 时间片长度与优先级成反比
- Définis statiquement pour chaque niveau de priorités

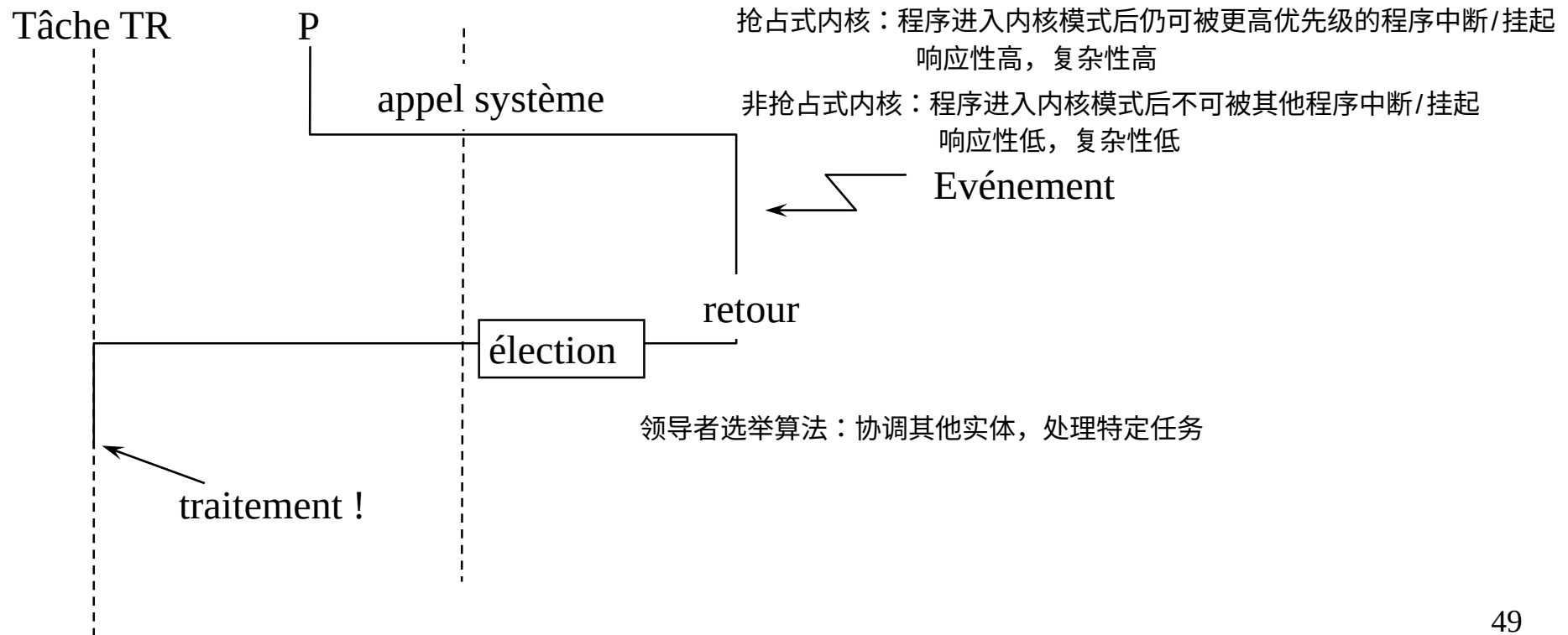
时间片不随系统动态变化而变化

pri	quantum	pri suiv.	maxwait	pri wait
0	100	0	5	10
1	100	0	5	11
...	...	...	...	...
15	80	7	5	25
...	...	...	...	...
40	20	30	5	50
...	...	...	...	...
59	10	49	5	59

优先度为0的程序，在度过长度为100的时间片之后，优先度变为0，

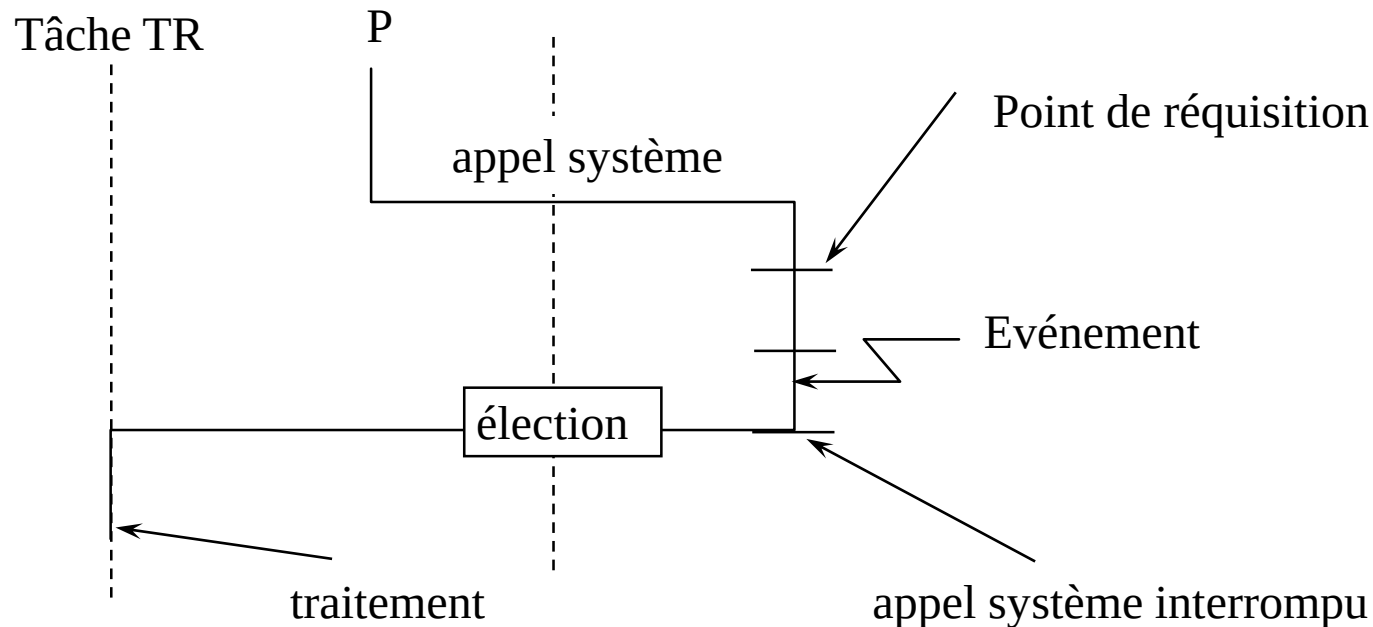
Classe "temps réel"

- Objectif : satisfaire des contraintes de temps
 - Processus temps réel très prioritaire en attente d'événement
- Impossible dans la plupart des Unix car noyau non-préemptif !



Points de réquisition

- Solution 1 : vérifier *régulièrement* si un processus plus prioritaire doit être exécuté (SVR4) 定期检查



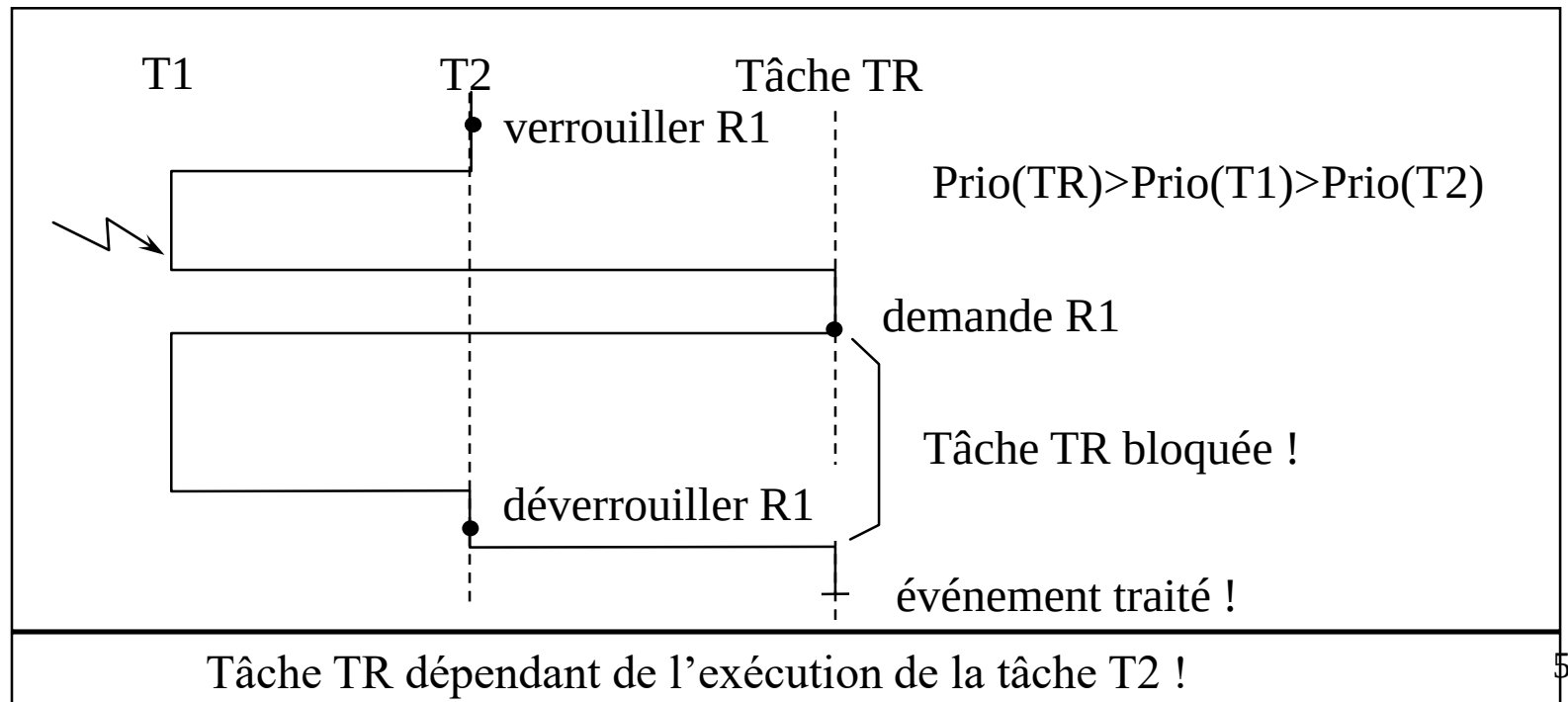
- Pratiquement il est difficile de placer de nombreux points
=> latence de traitement importante

Noyaux Préemptifs (Solaris 2.x)

- Solution 2 : rendre le noyau préemptif 抢占
=> en mode noyau l'exécution peut être interrompue par des processus plus prioritaires
- Protéger toutes les structures de données du noyau par des verrous (~ sémaphores)

Inversion de priorité

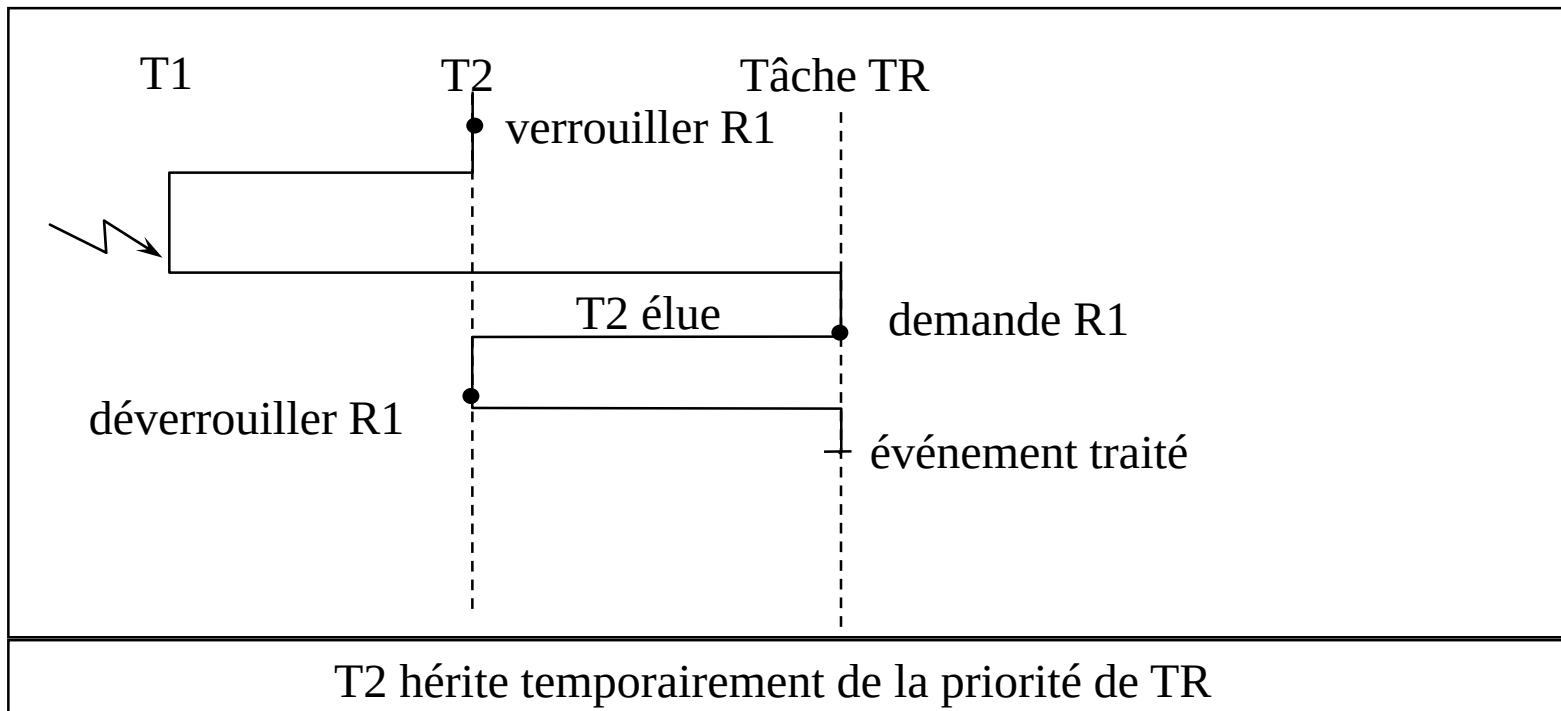
优先级翻转



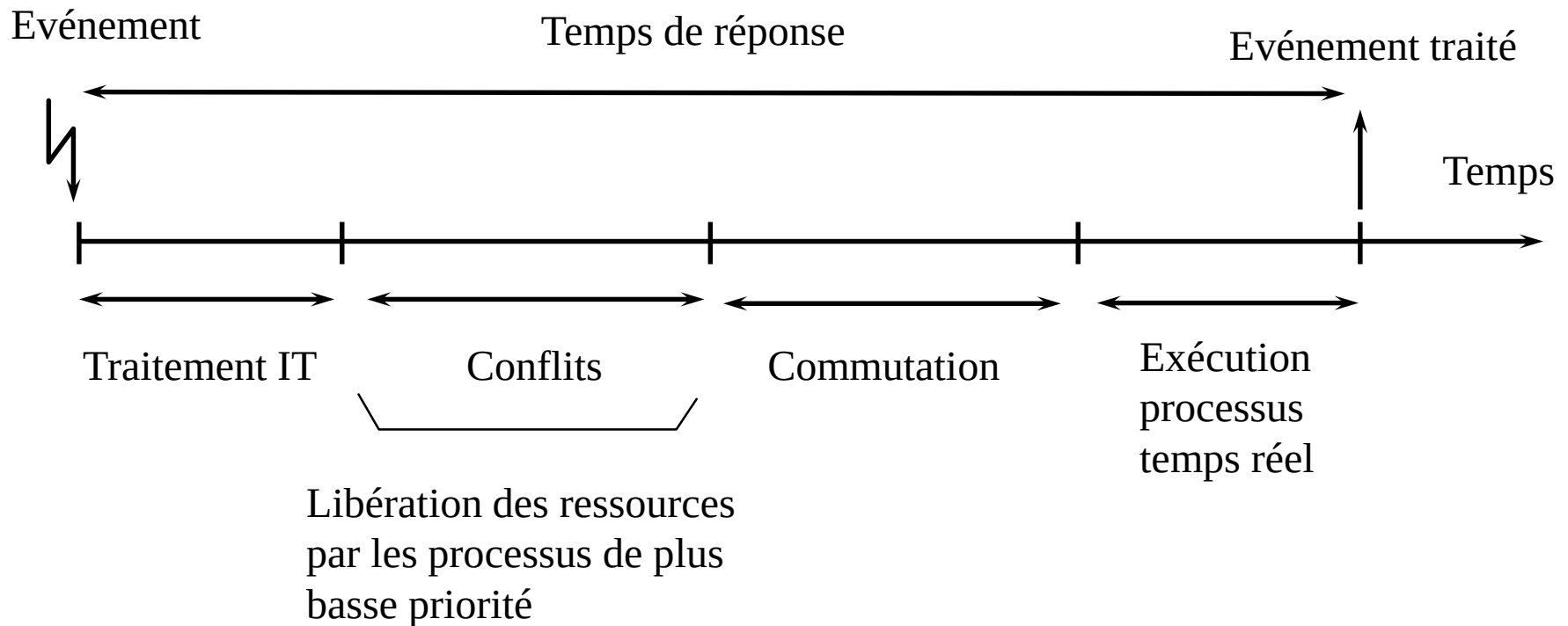
Héritage de priorité

- Solution : Donner à la tâche qui possède la ressource la priorité de la tâche temps réel

当优先级 $A > B > C$ ，且B正在运行，A需要C的文件，C锁死了文件，这时A会将优先级继承给C，来使A可以进行任务。



Temps de réponse



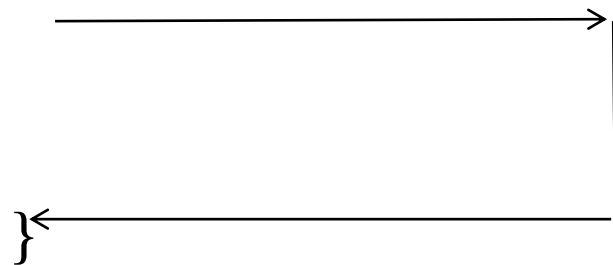
Appels systèmes liés aux processus

- `int fork()` : création d'un fils par **duplication** du père
- `exec(...)` : **recouvrement**, changement de code de l'appelant
- `exit(int)` : fin de l'appelant
- `int wait(int *)` : attendre la fin d'un fils 调用父进程等待子进程结束

Père

Fils

```
if (fork() == 0) {
```



```
[ execl("/bin/ls", "ls", "f1", NULL); ]  
exit(10);
```

```
}  
p= wait(&e);
```

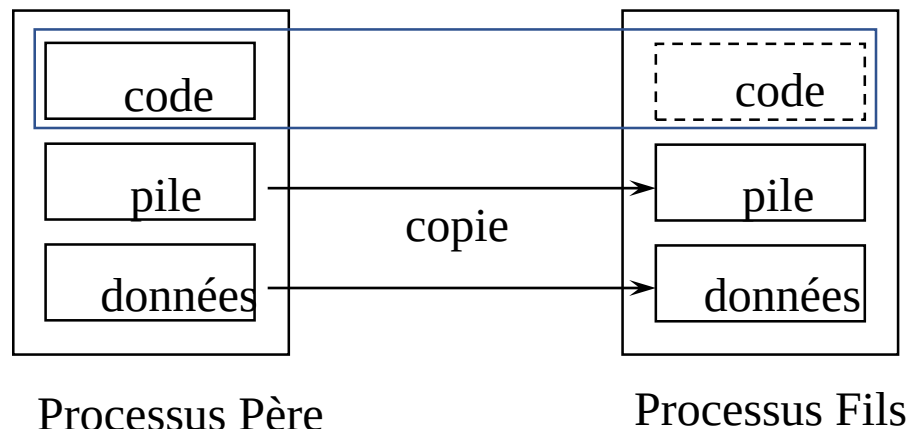
Fork : création d'un processus

- Création dynamique d'un nouveau processus (*fils*) qui s'exécute de façon concurrente avec le processus qui l'a créé (*père*).

`pid_t fork (void )`

创建子进程，二者并列运行

- Processus fils est **une copie du processus**
 - variables du fils possèdent les mêmes valeurs que celles du père au moment du *fork* 变量相同
 - toute modification d'une variable par l'un des processus n'est pas visible par l'autre. 对一个的变量进行修改，另一个不可见



Fork : création d'un processus

- Chaque processus reprend son exécution à l'instruction suivant le fork
- Valeurs de retour différentes selon le processus
 - **0** : renvoyé au processus fils
 - **pid (>0) du processus fils** : renvoyé au processus père
 - **-1** : appel à la primitive a échoué
 -
- `pid_t getpid(void);`
 - obtenir son pid
- `pid_t getppid (void);`
 - obtenir le pid du père

Terminaison d'un processus : exit

```
void exit(int val);
```

val = valeur récupérée par le processus père

⇔ return (val) dans le main

已执行完毕，但是父进程未读取其结束状态

- Le processus est détruit en mémoire et devient **Zombie**:
 - Etat d'un processus terminé tant que son père n'a pas pris connaissance de sa terminaison (wait).

Synchronisation père/fils – Wait

```
#include <sys/types.h>
#include <sys/wait.h>

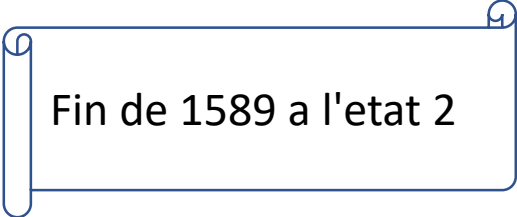
pid_t wait (int* status);
```

Attendre la fin d'un seul fils quelconque

- Argument et valeur de retour :
 - Valeur de retour : **pid** du processus fils terminé, retourne directement -1 si aucun fils.
 - Paramètre de sortie **status** passé par référence : état du processus fils (valeur d'exit).
- Macros définies dans <sys/wait.h> manipulant le paramètre **status** :
 - WEXITSTATUS(status) : valeur de retour ;
 - WIFEXITED(status) : valeur ≠ 0 si fin normale.

Wait – exemple

```
int main() {
    pid_t p, d;
    int e;
    p = fork();
    if(p == 0){ /* le fils */
        exit(2);
    } else { /* le pere */
        d = wait(&e);
        if(WIFEXITED(e)){
            printf("Fin de %d a l'etat %d \n", d, WEXITSTATUS(e));
        } else {
            printf("Probleme fils %d\n",d);
        }
    }
    return 0;
}
```



Fin de 1589 a l'etat 2

Wait – exemple zombie

```
int main() {  
    pid_t p, d;  
    int e;  
    p = fork();  
    if(p == 0){ /* le fils */  
        exit(0);  
    } else { /* le pere */  
        sleep(30);  
        wait(NULL);  
    }  
    return 0;  
}
```

*Créer un processus zombie pendant
environ 30 secondes*

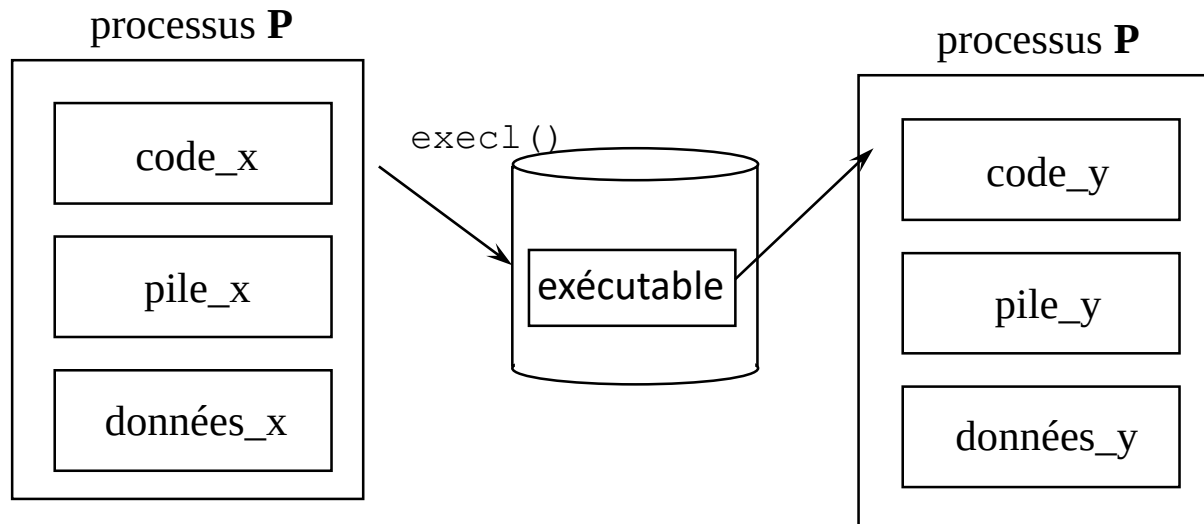
Remarque : Si le père est terminé, ses fils (orphelins)
sont “adoptés” par *init* (processus1).

Recouvrement - exec

- Le recouvrement permet à un processus de changer (écraser) son code et ses variables (pile et données)

=> le processus va exécuter **un nouveau main**. Si pas d'erreur, un exec ne retourne pas

Ne créer pas de nouveau processus (écrasement du contenu)



Le processus conserve : uid, gid, pid, ppid,
terminal, fichiers ouverts, signaux masqués

Primitives exec

 `int execl(char * pathname, char * arg0, ... , NULL);`

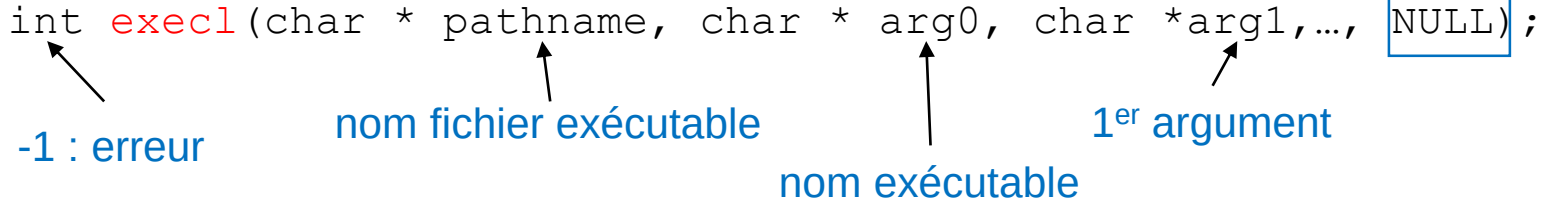
 `int execv(char * pathname, char * argv[]);`

 `int execlp(char * filename, char * arg0, ... , NULL);`

`int execvp(char * filename, char * argv[]);`

Exec - list

```
int execl(char * pathname, char * arg0, char *arg1,..., NULL);
```



-1 : erreur nom fichier exécutable nom exécutable 1^{er} argument

exemple : créer un processus qui exécuté "ls -l f1"

```
1.  if (fork()==0) {  
2.      execl("/usr/bin/ls", "ls", "-l", "f1", NULL);  
3.      perror("Erreur exec");  
4.      exit(1);  
5.  }
```

execlp identique avec l'exécutable recherché dans *PATH*

```
2.      execlp("ls", "ls", "f1", NULL);
```

Exec - vecteur

```
int execv(char * filename, char * argv[]);
```



-1 : erreur nom fichier exécutable vecteur d'arguments

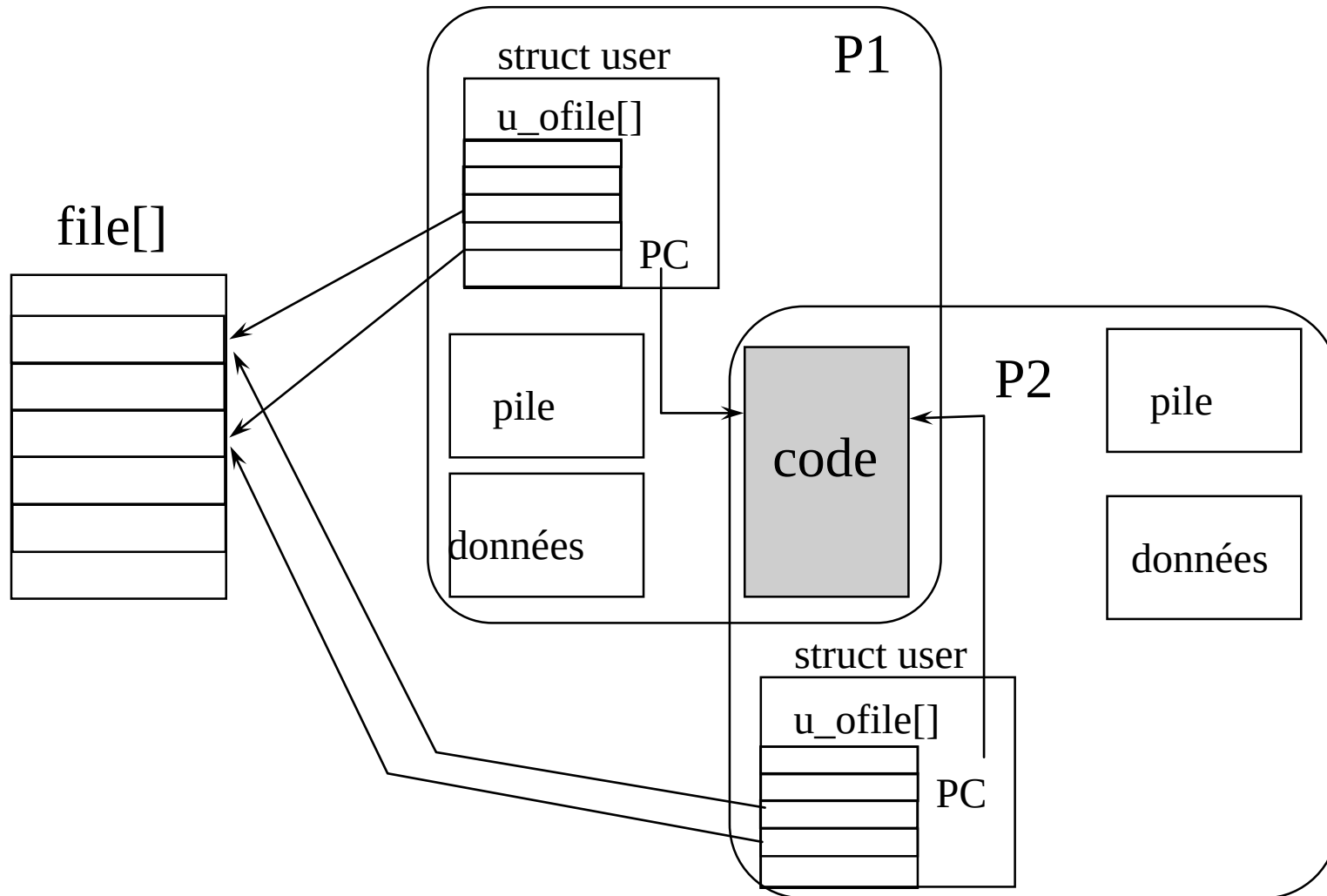
exemple

```
1.  if (fork()==0) {  
2.      char *argfiles[] = {"ls", "-l", "f1", NULL};  
3.      execv("/usr/bin/ls", argfiles);  
4.      perror("Erreur exec");  
5.      exit(1);  
6.  }
```


Fork : Création

- Fork : Créer un fils à l'image du père
 1. Réserver espace de swap
 2. Allouer un nouveau PID
 3. Initialiser struct proc
 4. Allouer tables zone de mémoire virtuelles
 5. Allouer struct user (copier du père)
 6. Mise à jour struct user
 7. Augmenter le nombre de processus partageant le code
 8. Dupliquer données + piles du père
 9. copier le contexte matériel du père
 10. Mettre le fils à l'état prêt + insertion dans la file
 11. retourner 0 au fils
 12. retourner nouveau pid au père

Création (2)



Exec : invocation d'un nouveau programme

1. Vérifier le nom de l'exécutable et si l'appelant a les droits d'accès
2. Lire l'entête et vérifier si l'exécutable est valide
3. Si le fichier a les bits SUID ou SGID positionnés, affecter les UID ou GID effectifs au propriétaire du fichier
4. Copier les arguments et variables d'environnement dans le noyau
5. [Allouer espace de swap pour les données et pile]
6. Libérer l'ancien espace d'adressage et les zones de swap associées
7. Allouer tables pour code, données et piles
8. Initialiser le nouvel espace d'adressage. Si le code est déjà utilisé le partager
9. Copier les arguments et l'environnement dans espace utilisateur
10. Effacer les routines de traitement de signaux définies.
11. Initialiser le contexte matériel (registres)

Remarque : les masques de signaux restent valides, les fichiers restent ouverts (sauf flag O_CLOEXEC dans Unix récents)

Exit: Terminaison

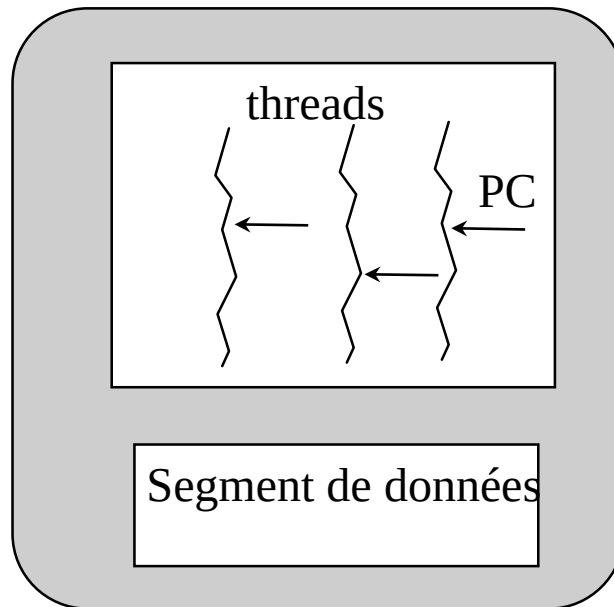
1. Annuler tous les temporisateurs en cours
2. Fermer les descripteurs ouverts
3. [BSD: Sauver la valeur de terminaison dans le champs p_xexit de struct proc
4. Sauver les statistiques d'utilisation dans champs p_ru]
3. Unix TD – V6 :
4. sauvegarde informations struct user dans le swap
5. Changer le processus à l'état SZOMB et mettre le processus dans la liste des processus zombies.
6. Libérer l'espace d'adressage, zone u, tables de pages, espace de swap
- [7. Envoyer le signal SIGCHLD au père (ignorer par défaut)]
10. Réveiller le père s'il était endormi (wakeup)
11. Pour tous ses fils, les faire adopter par init (ppid = 1) 便于回收
12. Appeler swtch() pour élire un nouveau processus

轻量级进程
介于进程和线程之间
共享相同的地址空间/资源

Processus légers

线程共享相同的地址空间
进程不共享

- Motivations :
 - 1) avoir une structure plus légère pour le parallélisme
 - 2) partage de données efficace



thread = code + pile + registres

线程是执行程序的基本单位，由它要执行的代码、它的私有堆栈以及它在任何给定时间点的CPU寄存器的状态组成

- thread (processus léger) : unité d'exécution

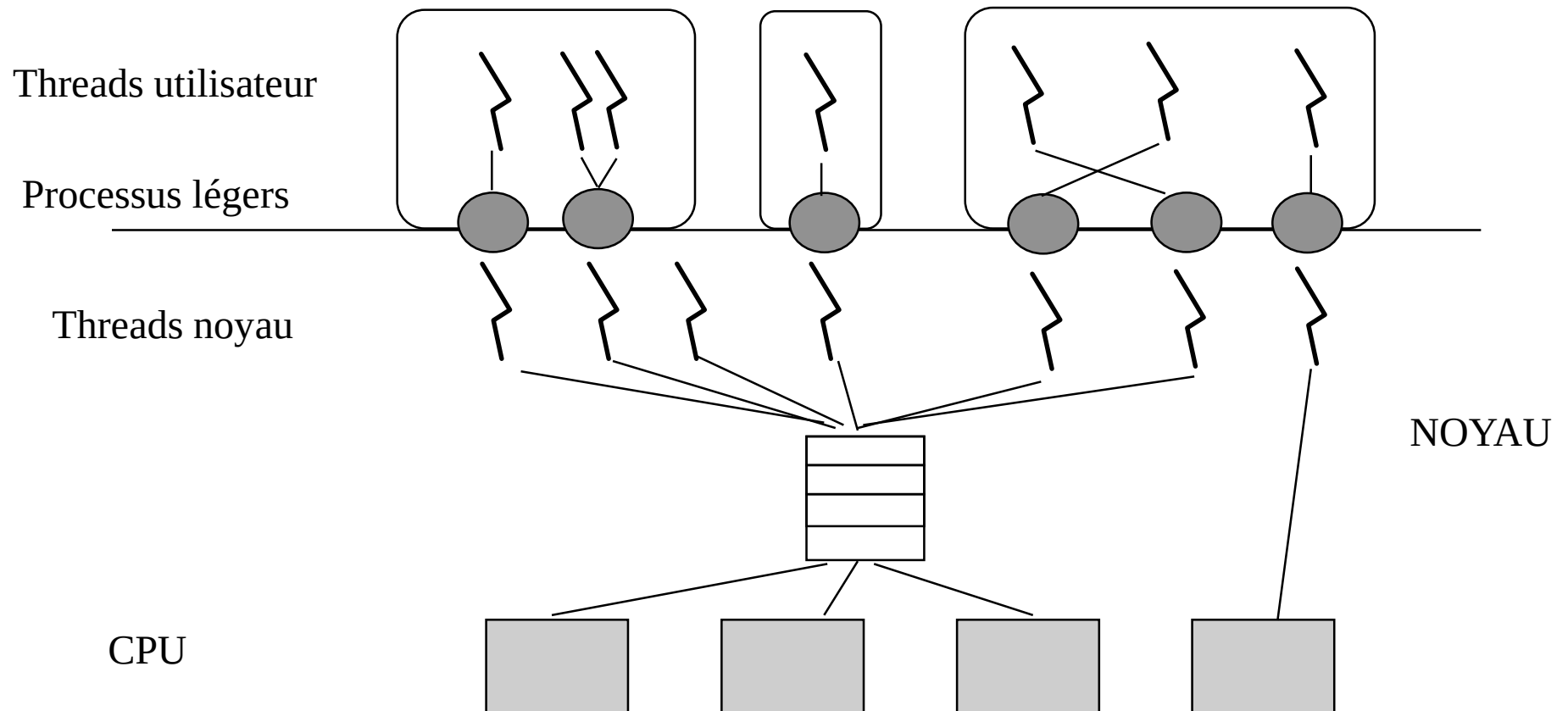
Propriétés des threads

- Partage le même espace => commutation plus rapide
- Echange de données par simple échange de référence
- Création/synchronisation plus rapide
- 3 types de threads (Solaris 2.x)
 - thread noyau : unité d'ordonnancement dans le noyau 调度和执行的基本单位
 - processus léger (lightweight process LWP) : associé à un thread noyau 用户级线程和内核级线程之间的中介
 - thread utilisateur : multiplexé dans les LWP

这些线程是在用户空间中创建和管理的，而不是在内核中。

每个LWP都与一个内核线程相关，与一个/多个用户线程相关

Exemple Solaris 2.x



Comparaison

	Temps de création (microsecondes) 微秒
Thread utilisateur	52
Processus léger	350
Processus	1700

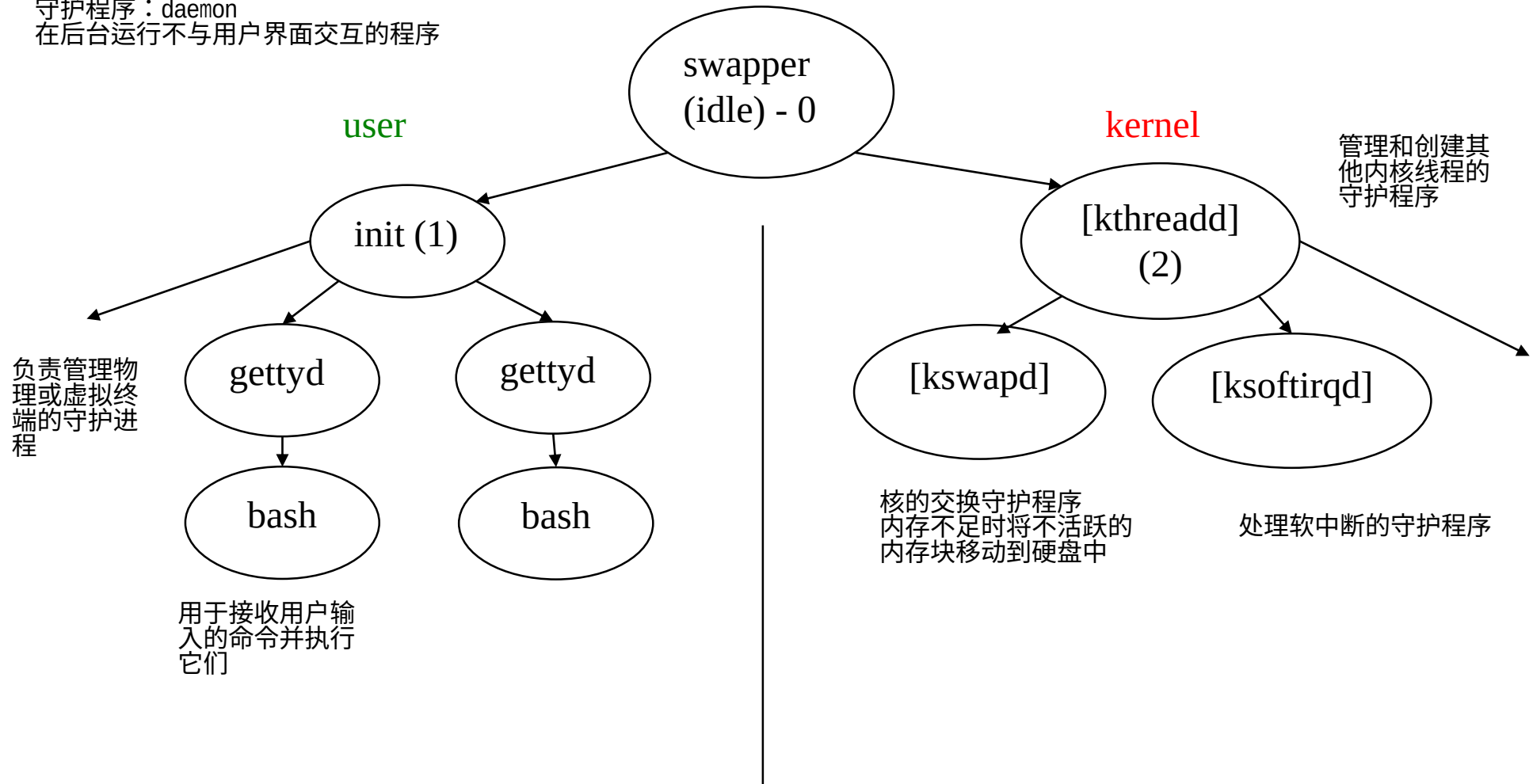
Solaris sur Sparc2

Gestion des processus dans LINUX

- Structures
- Ordonnancement
- Nouveautés depuis 2.6

Arbres des processus

守护程序：daemon
在后台运行不与用户界面交互的程序

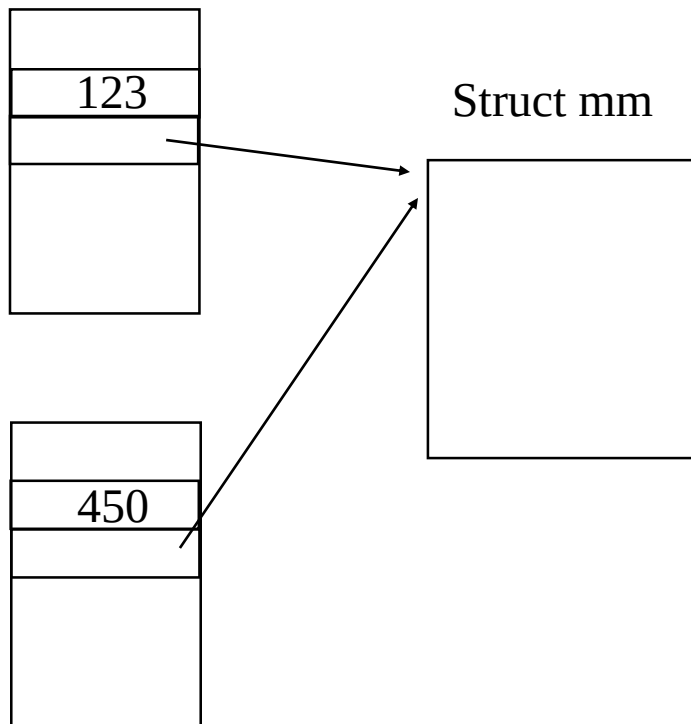


Structure de données : struct task

- 1 table des processus de type struct task (equivalent à proc + user)
- 1 entrée par processus
- Première entrée réservée au processus **init**
- **Task_struct :**
 - **policy** (SCHED_OTHER, SCHED_FIFO, SCHED_RR) : stratégies d'ordonnancement 调度策略
 - **state** : running, waiting, stopped, zombie
 - **priority: quantum de base**
 - **counter** : compte le nombre de tics restants avant la prochaine commutation
 - **next_task, prev_task : liste** 可以被组织成一个双向链表, 从而方便地在系统中遍历所有进程。
 - **mm_struct : contexte mémoire**
 - **pid, pid : identifiant**
 - **fs_struct : fichiers ouverts**
 - ...

Threads

- Implémentation des threads dans la noyau :
 - Simple partage de la structure struct mm



Processus Linux (≤ 2.4)

- **Trois classes de processus**

- Processus interactifs : attente événement clavier/souris, temps de réponse court
交互式进程
- Processus « batch » : lancement en arrière plan, plus pénalisé par ordonnanceur
批进程
- Processus temps-réel : forte contraintes de synchronisation (multi-média, commandes robots ..)
实时进程

进程状态

- **Etats :**

- Running
- Waiting
- Stopped
- Zombie

Stratégies d'ordonnancement (≤ 2.4)

- Noyau **non-preemptif** mais ordonnancement **preemptif (quantum)**
非抢占式内核，但是执行抢占式策略
- **Tic = 10ms (paramètre HZ = 100 défini dans param.h)** 时钟中断10ms一次
- **Deux types de priorité correspondant à 2 classes d'ordonnancement :**
 - 静态优先级：由管理员安排
 - **Priorité statique** : processus temps-reel (1 à 99), priorité fixe donnée par l'utilisateur
 - **Priorité dynamique** : somme de la priorité de base et du nombre de tics restants (counter) avant la fin de quantum
 - 动态优先级：随时间而变化

Algorithme d'ordonnement

- Temps divisé en **périodes (epoch)** epoch : 时间段
- Début période :
 - Un quantum associé à chaque processus prêt
- Fin période :
 - Tous les processus ont terminé leur quantum
- Calcul du quantum :
 - 1 quantum de base = 20 tics (200 ms) — 时间片 200ms
 - `#define DEF_PRIORITY (20*HZ/100)` (=20) 定义默认优先级为20
 - `priority = DEF_PRIORITY`
 - Counter : temps restant (nb tics) 剩余时间, 单位tics (时钟中断时间)
 - Création : le processus hérite de la moitié du quantum restant du père 子程序可以继承父程序一半长度的时间片
- Champs priority et counter pas utilisés pour les processus de classe SCHED_FIFO
 - 调度策略, 一旦运行无法被抢占

Fonction schedule()

- Implémente l'ordonnancement

- Invoquée directement en cas de blocage 死锁直接调用
- Invoquée « paresseusement » au retour en mode U

- Schedule

1. Choisir le meilleur candidat : celui ayant le poids le plus élevé (fonction goodness) :

Poids = 1000 + priorité base pour processus temps réel

Poids = counter + priorité base pour autre processus

Poids = 0 si counter = 0

2. Si tous les processus prêts ont un poids de 0 => **Fin de période**

1. Ré-initialisation des counter de TOUS les processus :

- $p \rightarrow \text{counter} = (p \rightarrow \text{counter} \gg 1) + p \rightarrow \text{priority}$ 新的 p->counter 值等于当前 p->counter 值的一半加上进程的基本优先级
- Rem : la priorité des processus en attente augmente

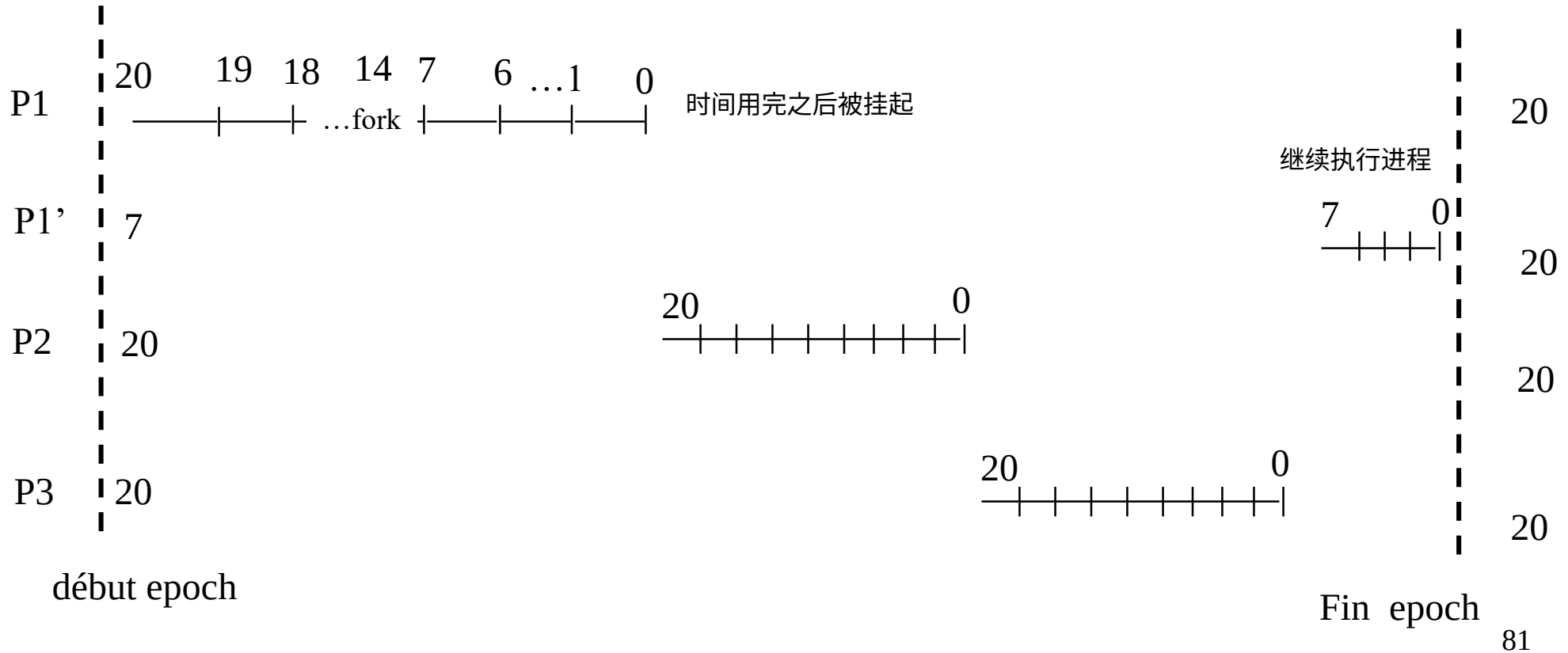
等待进程的优先级提高

Epoch是一组时间片的集合，是用于评估和调整进程优先级的时间窗口。在一个Epoch内，每个进程都获得一个基于其优先级的时间片。当Epoch结束时，系统会重新评估每个进程的优先级，并为下一个Epoch分配时间片。

Exemple

- Evolution du champs *counter*

数字：进程计数器的值



Ordonnancement SMP (1)

- Critère supplémentaire pour l'ordonnanceur :
 - Moins coûteux de ré-exécuter un processus sur le même processeurs (exploitation des caches internes) 优先使用同一个处理器
 - Maximiser l'utilisation des différents processeurs 最大化使用不同的处理器
- Exemple :
 - 2 processeurs (CPU1, CPU2) et 3 processus (P1, P2, P3)
 - Priorité $P1 < \text{Priorité de } P2 < \text{Priorité de } P3$
 - CPU1 exécute P1
 - CPU2 exécute P3
 - P2 exécution précédente sur CPU 2 devient prêt
 - Question : P2 « prend » CPU1 (préemption) => perte du cache de CPU2 ou attendre que CPU2 deviennent disponible ?
如果P2在CPU1上
加载缓存的时间
小于等待P3的时间，
就抢占CPU1
- => Heuristique qui prend en compte la taille des caches

Ordonnancement SMP (2)

- P2 préempte P1 sur CPU1 si :
 - Le quantum restant de P3 sur CPU2 (counter) est supérieur au temps estimé de remplissage des caches de CPU1

Nouveautés Linux 2.6

- Ordonnancement
- Noyau préemptif

Ordonnancement 2.6.X

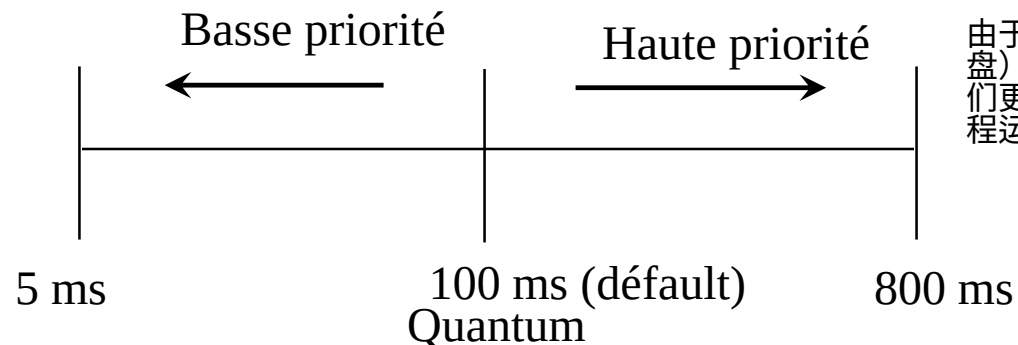
- Objectif : diminuer les temps de réponses et une gestion plus fine des temporisateurs pour applications multi-média

=> diminution de la valeur du tic (jiffy) = 1 ms (HZ =1000)

- 2 types de processus

- I/O Bound (E/S) : processus faisant beaucoup d'E/S 输入输出进程
- Processor Bound : processus de calcul 计算进程

- Objectif : avantager les processus I/O Bound avec des quantum variables



由于I/O Bound进程经常被外部资源（如磁盘）阻塞，给它们更短的时间片可以使它们更快地返回等待队列，从而允许其他进程运行。

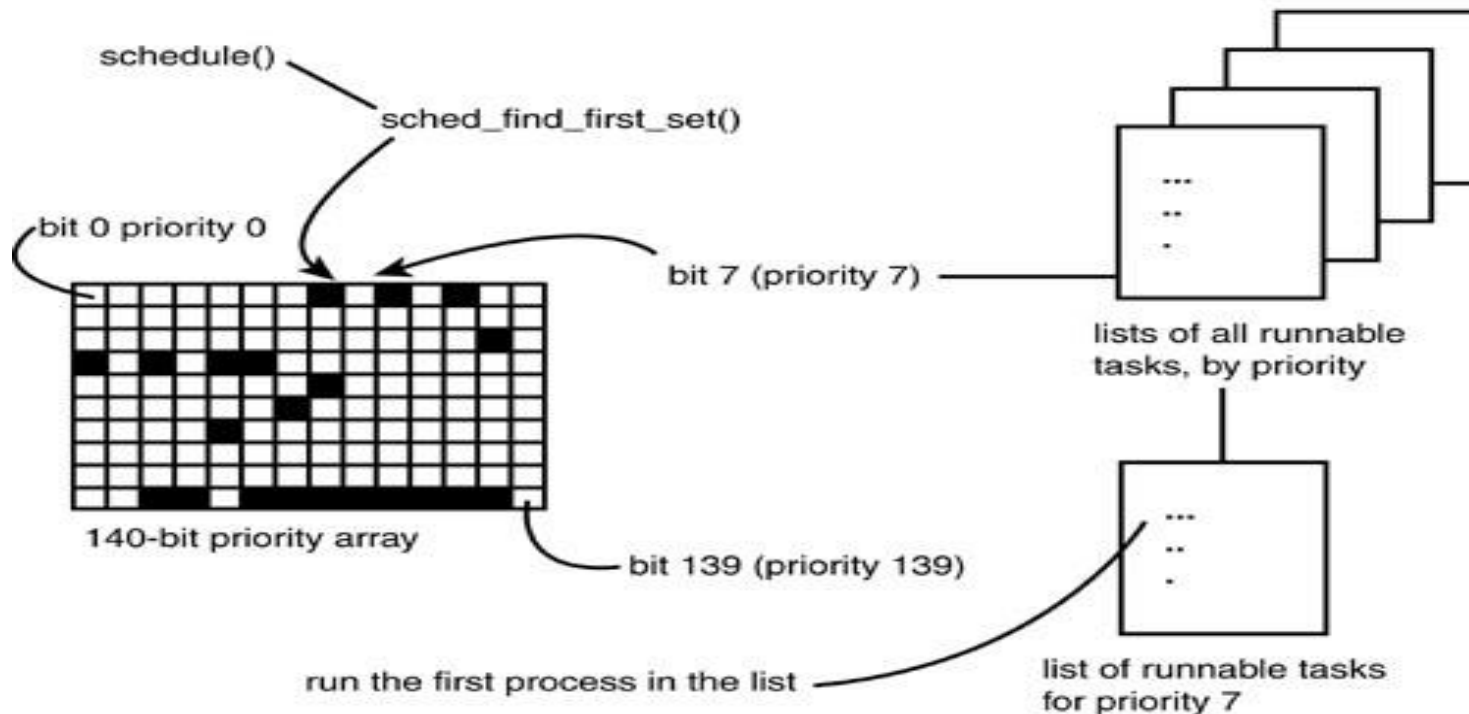
Algorithme d'ordonnancement 2.6 : Priorité

- Priorité de base = valeur du nice [-20,+19] 优先级在-20到19之间
- Système de pénalité/bonus en fonction du type de processus
Bonus max = -5, Penalité max= +5 奖：不常使用CPU
惩：常使用CPU
- Pour déterminer les types de processus : ajout d'un champs sleep_avg dans structure task sleep_avg：进程在“阻塞状态”下所花费的平均时间
- Sleep_avg = temps moyen à l'état bloqué
 - Au reveil d'un processus : sleep-avg augmenté
 - A l'exécution : sleep-avg-- à chaque tic 每过一个tic, sleep-avg递减
- Fonction effective_prio() : correspondance entre sleep_avg et bonus [-5,+5]

根据sleep_avg进行-5到+5的bonus

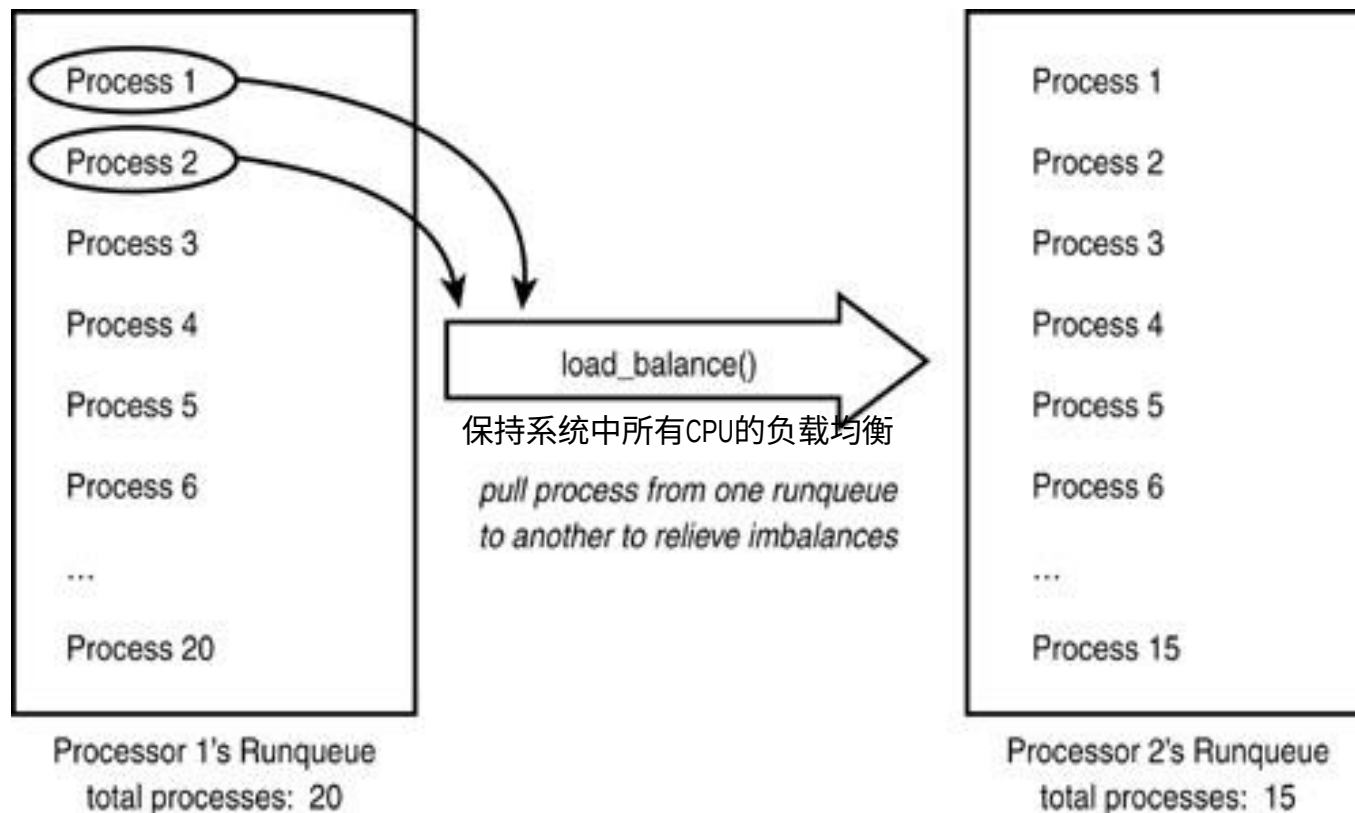
Algorithme de choix du processus

- Algorithme en $O(1)$
- 140 niveaux de priorité, 1 file par niveau



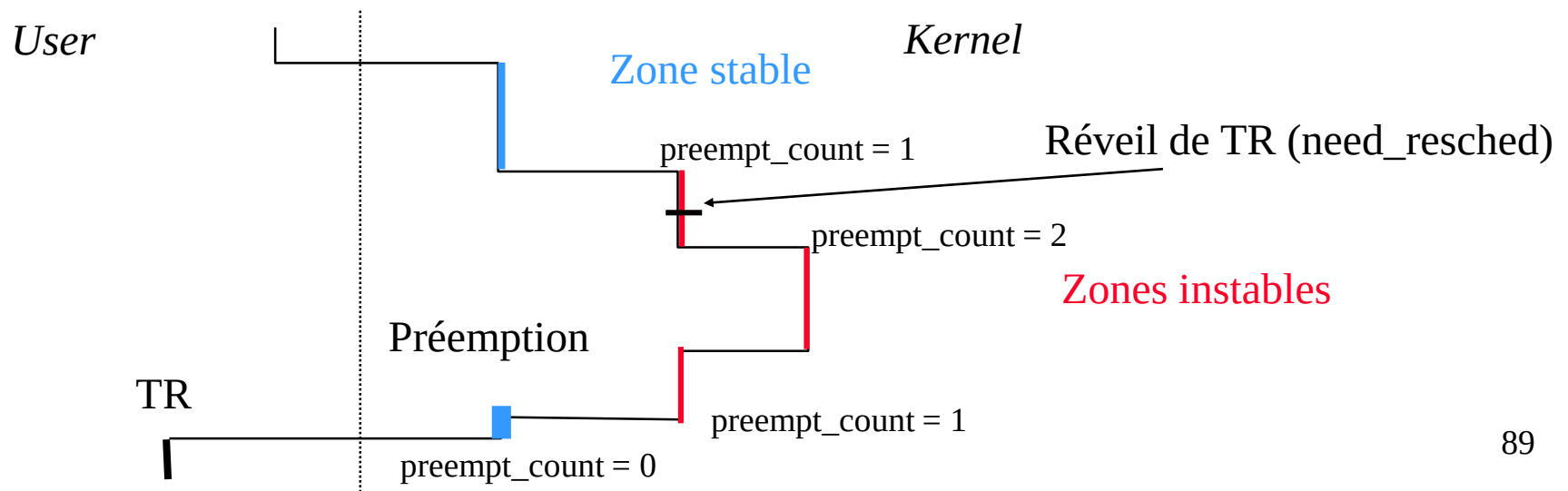
Equilibrage de charge

- Fonction `load_balance()` appelée par `schedule()` lorsqu'une file est vide ou périodiquement (toutes les ms si aucune tâche, toutes les 200 ms sinon)

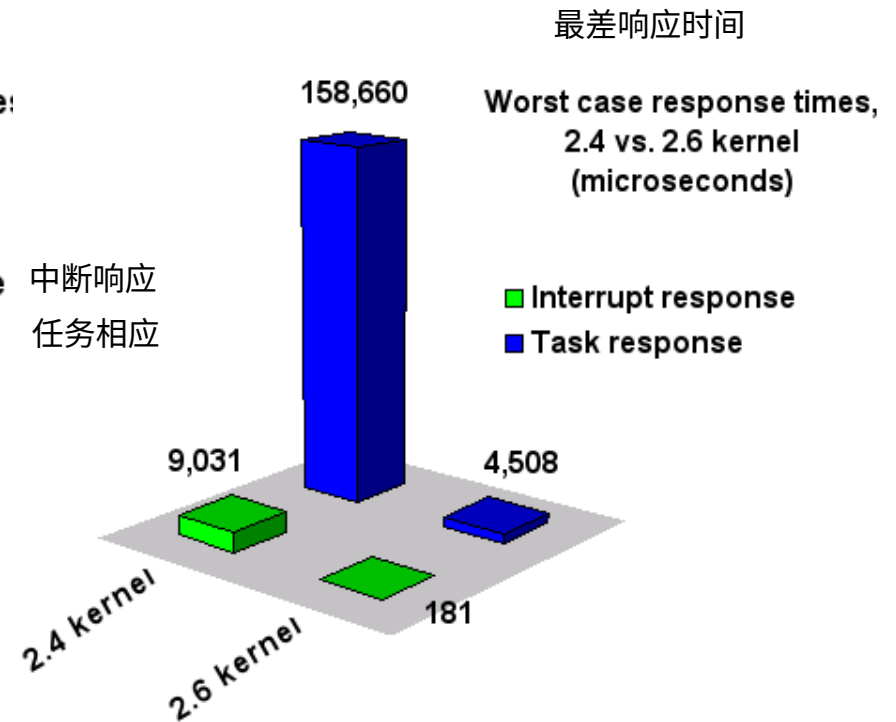
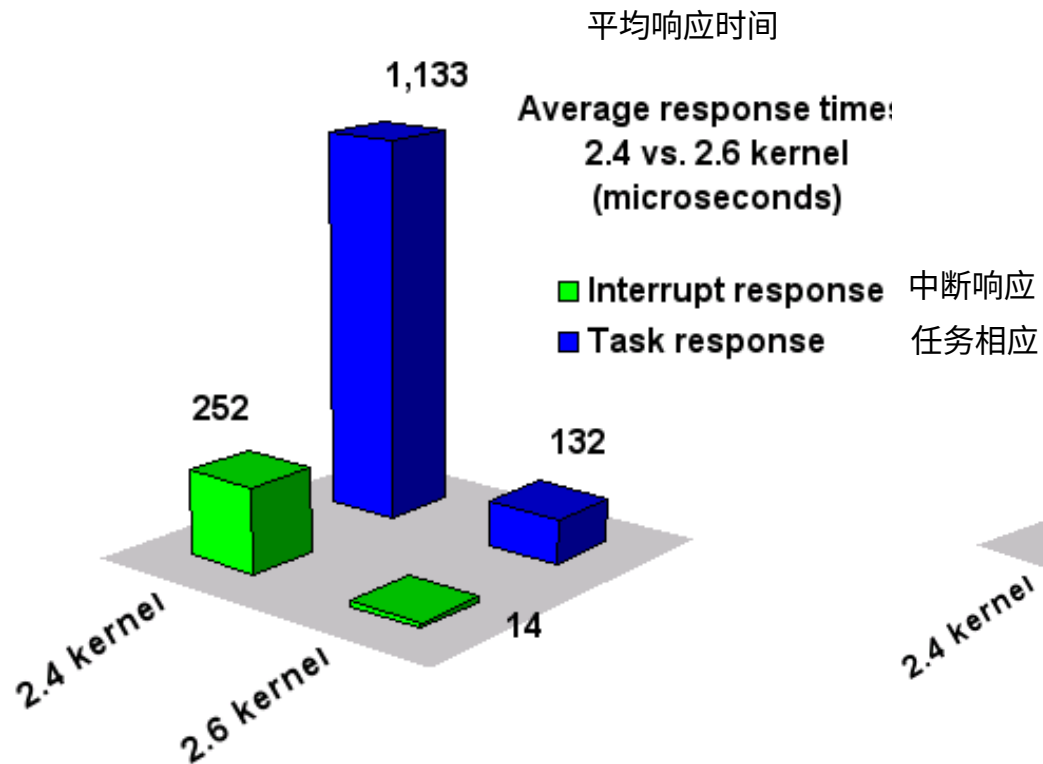


Noyau Préemptif – Linux 2.6.x

- Proche de la notion de points de réquisition : 只在特定的稳定点进行抢占
 - Quitter le noyau uniquement à des points stables
- Verrouillage pour protéger les régions instables :
 - => un compteur (preempt_count) incrémenté à chaque verrouillage
- Retour d'IT : 当need_resched标志被设置（意味着需要重新调度）并且preempt_count为0（当前没有任何锁被持有）时，当前任务可以被抢占。
 - si need_resched et preempt_count == 0 → Préemption



Linux 2.4 vs. 2.6



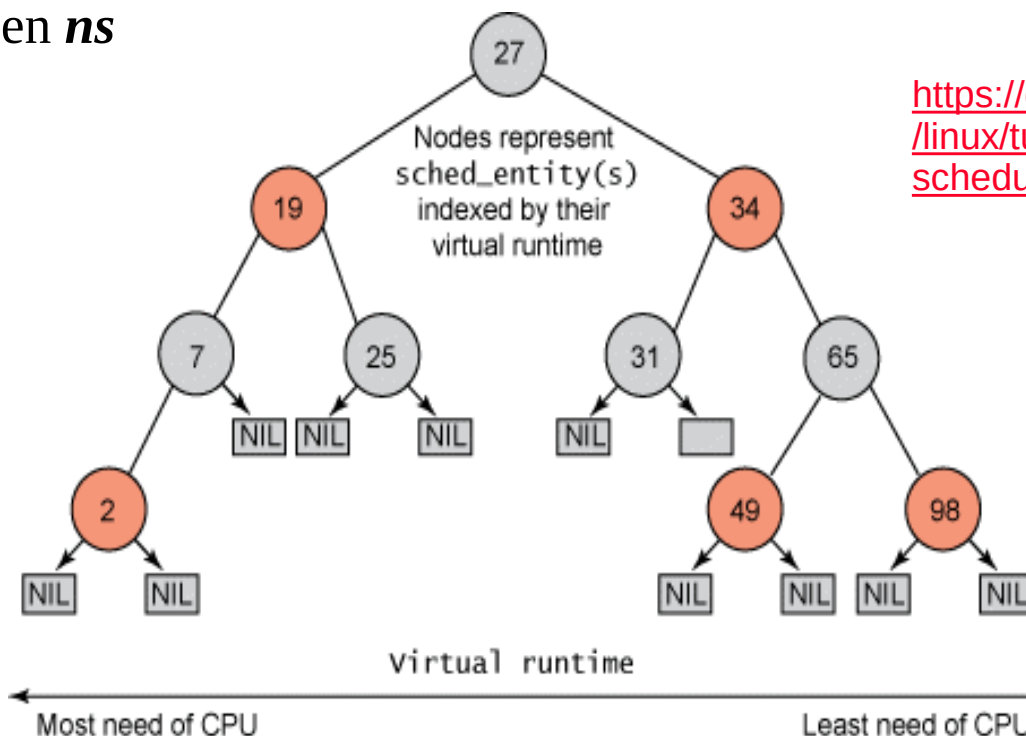
Linux 3.x - “Completely Fair Scheduler”

完全公平调度器

- Depuis Linux 2.6.23 un nouvel ordonnanceur : CFS
- Tâches prêtes stockées dans un arbre rouge-noire (~équilibré) => Temps de mise à jour en $O(\log n)$
- Tâches ordonnées en fonction de leur temps virtuel d'exécution (*virtual runtime*) exprimé en **ns**

任务在CFS中是根据它们的虚拟运行时间 (virtual runtime, 简称vruntime) 来排序的。

虚拟运行时间：一个表示任务已经运行了多久的度量，单位是纳秒。

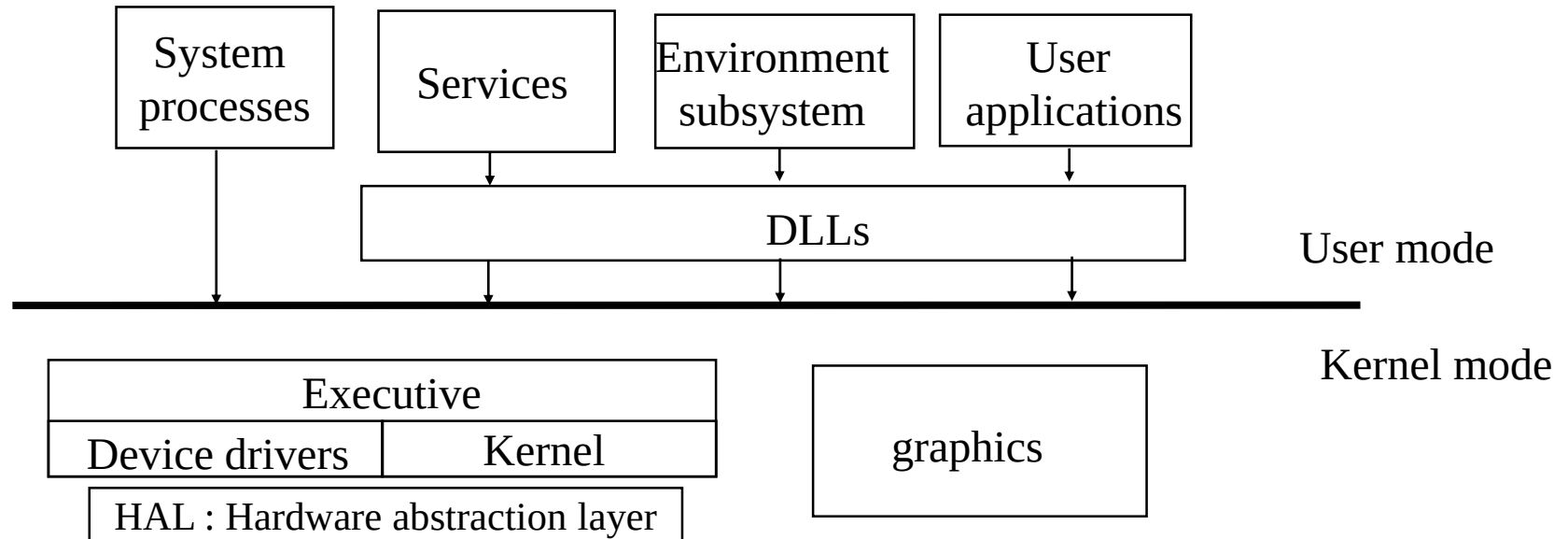


<https://developer.ibm.com/technologies/linux/tutorials/l-completely-fair-scheduler/>

CFS

- A l'invocation de l'ordonnanceur :
 - Choisir la tâche ayant **le plus petit virtual runtime** : (la tâche la plus à gauche dans l'arbre) => Tâches courtes ou qui ont fait des E/S sont avantagées
 - Calculer le temps d'exécution maximum pour le processus élu (fonction du nombre de tâches et de son temps d'attente)
 - Si le processus a atteint son maximum : mise à jour de son virtual runtime et réinsertion dans l'arbre, choisir un nouveau processus
- Intégration des priorités :
 - Augmentation de la priorité => diminution d'un certain facteur (decay) du virtual runtime
 - Virtual runtime est relatif

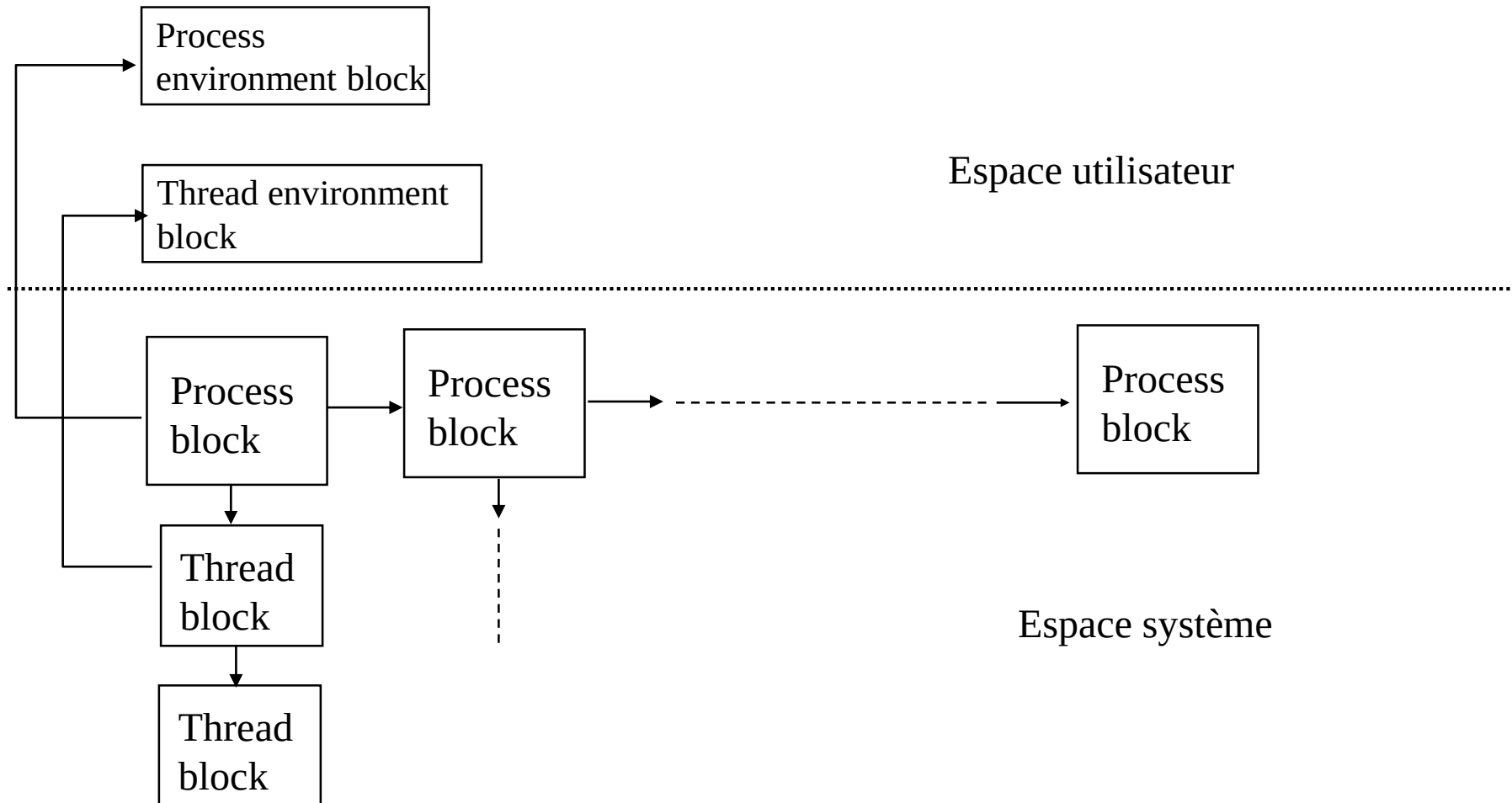
Gestion de processus dans Windows NT



1. Processus et Threads

2. Ordonnancement

Les structures

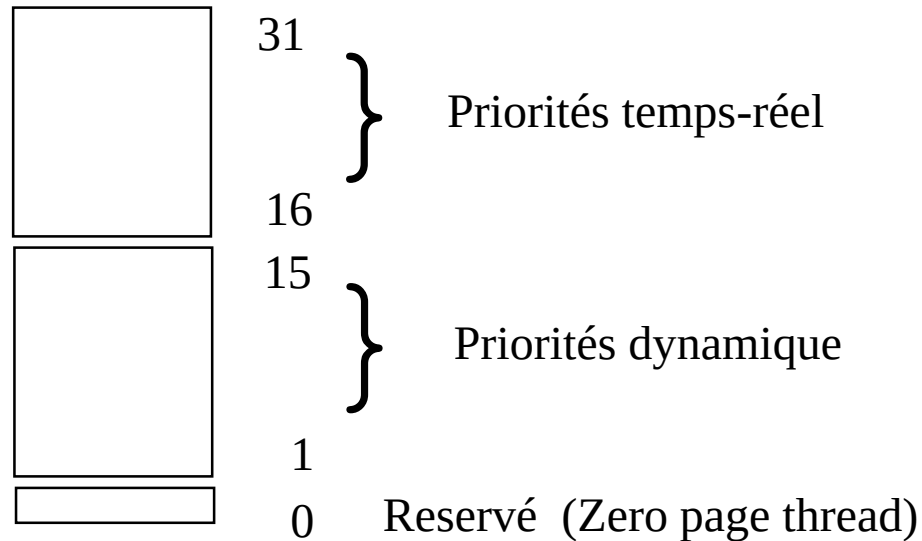


Les structures

- Process block (EPROCESS) : similaire à struct proc Unix
 - PID, PPID
 - Valeur de retour
 - Kernel process block (PCB) : statistiques, priorité, état, pointeur table des pages
- Thread block (ETHREAD) :
 - Statistiques, adresse de la fonction, pointeur pile système, PID ...
 - Kernel thread block (KTHREAD) : synchronisation, info ordonnancement (priorité, quantum ...)
- Process Env. block (PEB) :
 - Informations pour le « chargeur », gestionnaire de pile (modifiable par DLLs)
- Thread Env. block (TEB) :
 - TID, information pile (modifiable par DLLs)

Ordonnancement

- Priorités

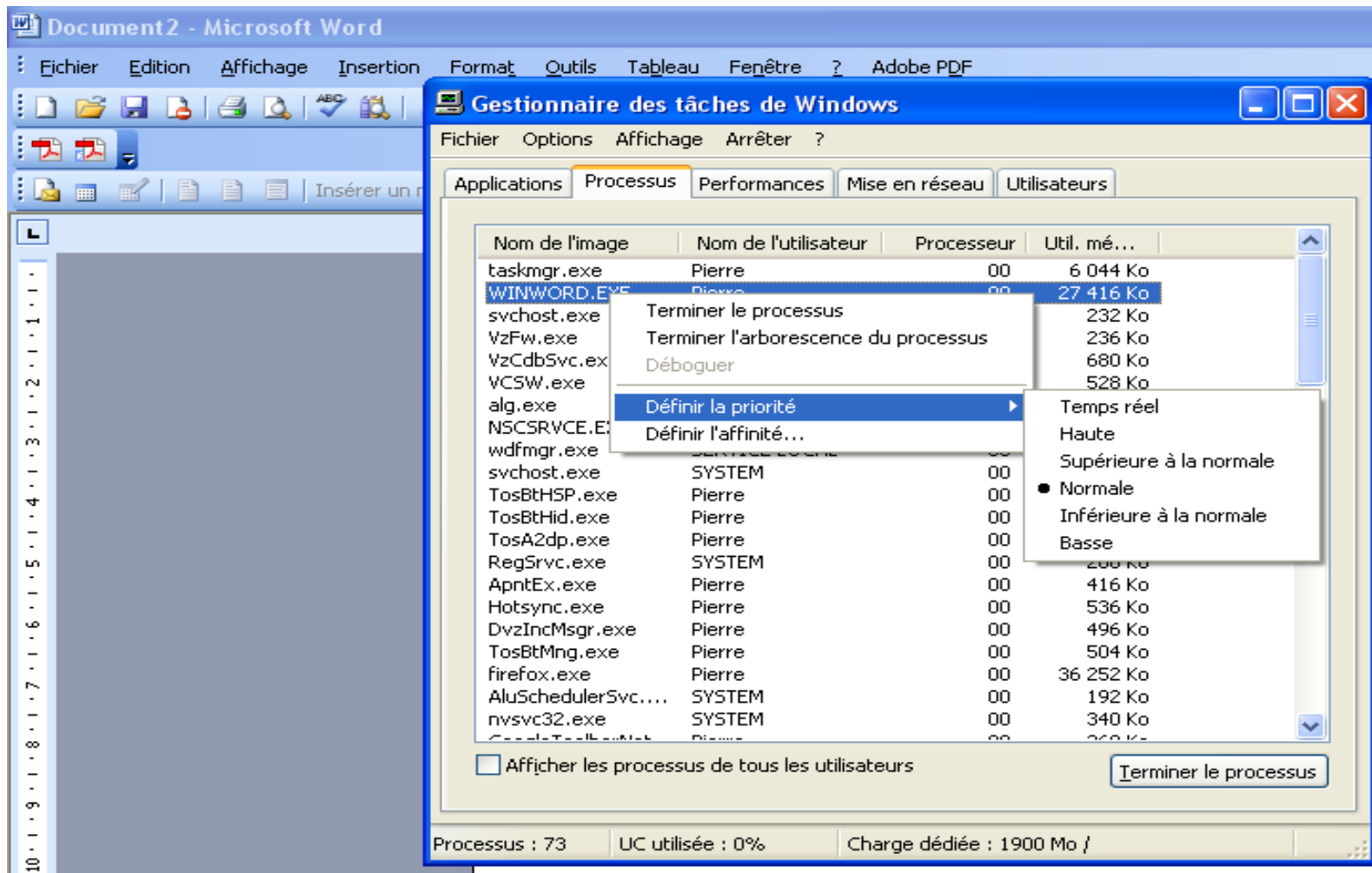


Ordonnancement temps partagé (quantum)

Par thread : priorité de base (processus), priorité courante

Choisir le thread le plus prioritaire (structure similaire à « whichqs » 4.4 BSB)

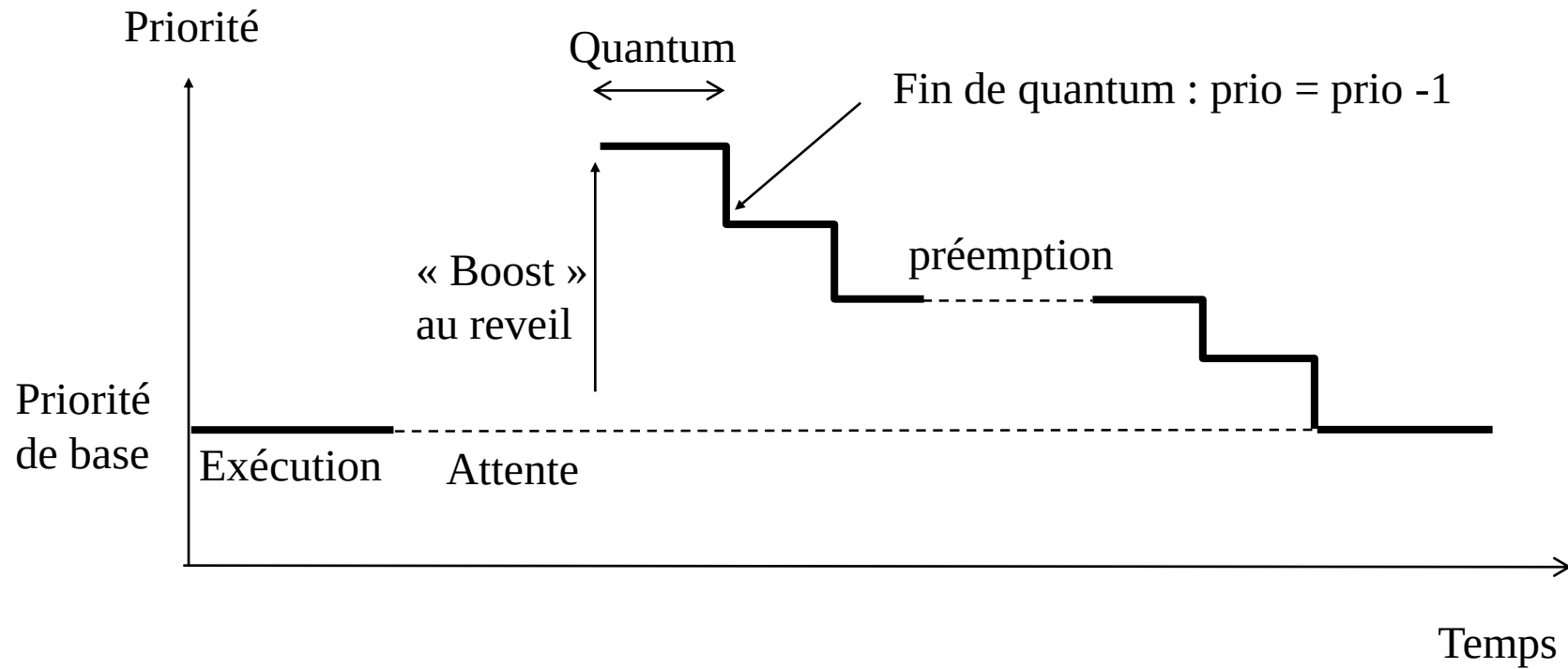
Classes de priorité



Ajustement des priorités et du quantum

- Création = priorité thread = priorité base (dépendant de la classe)
 - 4 mécanismes 增加后台进程的时间片
 - Augmentation du quantum des threads de processus en « arrière plan »
 - Augmentation de la priorité des processus endormis (*boost*)
 - +1 = sémaphore, disque, CR-ROM, port parallèle, vidéo 增加休眠进程的优先级
 - +2 = réseau, port série, tube
 - +6 = clavier, souris
 - +8 = son
 - Augmentation de la priorité des threads prêts en attente (éviter famine)
 - Thread en attente depuis 300 tics (~ 3 sec) 增加长时间等待的线程的优先级
- => priorité = 15, quantum x 2

Exemple



理想处理器：上次处理这个线程的处理器，可能仍有线程的缓存数据，可以提高性能

处理器亲和性：每个线程都有一个定义的“affinité”或亲和性列表，这个列表指定了线程可以在哪些处理器上运行。

Ordonnancement SMP

多处理器系统

- Définition d'**affinités** pour chaque thread : liste des CPU sur lesquels peuvent s'exécuter la tâche
- Chaque thread a un processeur “idéal”
- Quand un thread devient prêt, il s'exécute :
 - Sur le processeur idéal s'il est libre
 - Sinon sur le processeur précédent si il est libre
 - Sinon rechercher un autre thread prêt

调度决策：

当线程变得可运行时，系统会按照以下优先级来决定在哪个处理器上运行它：

- a. 如果线程的“理想处理器”是空闲的，那么线程就在那个处理器上运行。
- b. 如果理想处理器被占用，但线程之前运行的处理器是空闲的，那么线程就在那个处理器上运行。
- c. 如果上述两个选项都不可用，系统会在所有可用的处理器中查找一个空闲的处理器来运行线程